2023 NEA Project

By Erol Halim

Contents

Introduction..... 3
Who is this problem for..... 4
Statement of the problem..... 4
Problem with other adaptations..... 4
Context..... 4
Interview..... 5
Rules..... 6
Frameworks..... 6
Objectives..... 7
About this section..... 8
GUI vs Terminal..... 8
File Structure..... 15
Late Additions..... 16
Flow Charts..... 21
Code / Data Structures / Algorithms / OOP..... 22
Main.py..... 37
Login.py..... 39
Uno_cards.py..... 41
extra_module.py..... 53
Gui.py..... 54
The Game Loop..... 77
Setting up app/Sign in and Sign up (start loop)..... 82

UNO Cards and Extra..... 83
Objectives Completed..... 94
Feedback..... 97
Improvements for the future..... 99

Analysis

Introduction

For my programming project I have decided to create digital recreation of UNO with dynamic AI for the casual player. After several days of planning and discussions, I have fully finalised this idea. The objective of UNO is being the first person to lose all your cards. The game can be played with two plus different people. For the game, the other companies will be AI players that will have different aggressions, depending on these aggression there AI would either play with higher or lower difficulty. Every new game of UNO would play differently to the previous, as players' behaviours would always be changing. However, the most important part of good AI is a consistent one, but not predictable. The algorithm would be able evaluate all the cards that are currently playable. The algorithm will evaluate which cards are better to play at the moment using weights. These weights would be sorted highest to lowest. The higher weight is the better card. After an aggression algorithm is run to check whether the AI player is currently winning, losing or drawing compared from player to the next player. The AI is given a max choice of three cards to pick from out of the playable cards. Depending on aggression the odds of choosing a better card varies out of the three. If no playable cards are available then the AI will pick up a card. AI can also evaluate its best colour choice when playing wild cards.

Who is this problem for

My programming project is intended for recreational use and is aimed towards the average casual player. It can be used as a tutorial to help some new players understand rules of the game as the rules would be enforced by the code. After many discussions with different players of the game, as seen in the interview below this led to new discussions of ideas to incorporate into the application.

Statement of the problem

I chose to make a recreation of UNO, because of my own interest in the game, also due to that lack of other adaptations of the game. Most other adaptations of the game play with the official rules of UNO that are not as fun. Furthermore, it allows me to introduce my own interpretation of the game.

Problem with other adaptations

Majority of UNO players play with different rules to the official. Meaning there is only one type of deck and rules are not what people asked to play with. Another thing other adaptations lack is a leaderboard system. The closest thing to the leaderboard I found in my

research was either an arbitrarily points score you would earn after a game from how quickly you won or it would just measure the user wins. The two applications were the official UNO by Ubisoft and crazy 8 by Vitalii Zlotskii.

Context

UNO was a card game created in 1971. The objective of UNO is for the player to get rid of cards first, by playing them, while sabotaging others to make their progress slower. In UNO, there is only one winner.
UNO doesn't use typical playing cards, Instead, it uses a five-colour format with numbers 0,10 or symbols. Uno has to be played by more than 1 player but has no limit on how many players participate. Each colour of Uno cards can either be red, green, blue, yellow or wild with symbol cards having special abilities. An Uno deck has 112 cards, 76 of which are coloured, 8 skip cards, 8 reverse cards, 8 draw colours for each colour, 6 wild cards and 6 draws 4 cards. All coloured cards can only be played if the previous card's colour or number/symbol is the card the player is trying to play unless the card colour is wild. If the card colour is wild it can be played anytime. Symbol card reverse: reverse the current order of players e.g. if the order of players is clockwise (player1,player2,player3) if played it would be reversed to anti-clockwise (player3,player2,player1). Symbol card skip card when played skips the next player's turn and the player after gets to play e.g in the order was (player1,player2,player3) and if player 1 plays the card, player 2 does not get play as it skipped and it will now be player 3 turn. Symbol card draw, forces the player front to pick up two more cards. Wild draw cards force the player to pick up four cards and can change to become any colour. Wild cards can just change colour to become any colour. In this adaptation of Uno, players can only play one card at a time.

Interview

Ismail interview

This conversation took place before development of the game

Me: “As you know, I'm working on developing a new card game application, and I'd like to get your thoughts on what you would like to see in the game and what do you expect from this game?”

Ismail: “Well, first and foremost, I think it's important that the game has a clear and intuitive user interface. It should be easy to navigate and understand, even for someone who has never played the game before.”

Me:” That's a good point. I'm already planning on having all the game menus and options displayed on a GUI to make the experience more user-friendly. Are there any other suggestions you have in terms of user interface?”

Ismail: “Yeah, I think it would be helpful to have some visual cues to help players understand what moves are available to them. For example, if a player has a playable card in their hand, it could be highlighted”

James interview

This conversation took place mid development of the game

Me: "What are your thoughts on the game?"

James: "I think it would be great if there were different game modes available. For example, a timed mode where players only have a limited amount of time to make their move, or a mode where players can only play certain types of cards."

Me: " Yeah thanks for your input, James. I kinda like the idea of different game modes, and I'll definitely consider adding different game modes. It would add some variety to the gameplay and give players more ways to challenge themselves."

Rules

This version of Uno is a mix between the official rules, house rules and my own rules for the project:

- 4 Players are needed to start the game
- A deck shuffled so every card is in random position
- The order starts in clockwise direction e.g. player 1 to player 2
- The game starts at player 1
- Each player handed 7 cards from the deck
- Each player cannot see each others card
- Each player can only place one card for their turn
- A card can only be played if it share the same colour as the previous card or the same number/symbol or if it a wild
- Player that has no cards to play has to draw a extra card from the deck
- If a player draws a card it has the options to either try to play it if legal or keep it

- The reverse card switches the order of players until another player plays the card again
- The block card skips the next player's turn
- The plus 2 card forces the next player to draw two additional cards from the deck (however the player can still play in this version)
- The wild switch (or Wild) card allows the player to change the colour of the card on top and can be played anytime
- The wild plus four has the same properties as the wild switch but with adding effect of forcing the next player to draw four additional cards from the deck (however the player can still play in this version)
- The player in this adaptation doesn't have to say UNO when they have one card
- The game over when a player manages to remove all cards from their hand, and is the crowned the winner
- A draw can occur if all cards from the deck are drawn out, meaning there are no extra cards to be drawn.

Frameworks

For the game, I decided to code the majority of the full stack in python, due to its easy read syntax from the use of indentation, the use of interpreter so it can check code line by line, language being OOP based, furthermore it's a wide range of external/built-in modules for a vast range of different functions I could have access to. Parts of the back stack are made from SQL with the use of SQLlite3 module to allow easy manipulation of databases and complex queries. The use of SQL was to create relations to show hold users/player stats and to be updated after a game was created. The game front stack is created with the use of a python module named pygame. This module comes with a vast library of functions to help people create interactable video games in python. I used the module's function to be able to create a user-friendly GUI. GUI can restrict user choices and make programs more friendly to interact with. The use of clickable button objects and keypress allowed the game to become more accessible. The game also uses some other modules, for example, pygame text input by the team Nearoo on GitHub. This module gave me the ability to easily allow the user to enter text into a box and save it to a variable. This project would be programmed with the use of VScode editor for its auto-type features, shortcuts and its seamless collaboration with GitHub to be able to pull, commit and create a repository. This would also mean anyone with a computer can download the game off GitHub. The majority of assets are created with the use of photoshop for its flexible bitmapped and vector editing. The cards for the game were bought with a licence from Shutterstock.

Objectives

The project needs multiple objectives to aid in the development to guide a developer during the creation of the application.

- The application needs a login and sign-up system that is attached to a table that uses wins/losses/draws/games as its attributes.
- A database that is created if it doesn't exist
- All menu and options/gameplay must be displayed on a GUI to restrict user inputs and to create a more user-oriented experience
- The program needs a menu to allow users to play the game, view their stats and quit after they're done. This should be done with the use of buttons that are intractable with a mouse.
- The game needs AI players that are able to play rationally and are able to determine their options to be able to play the best card possible, but with a handicap of a bit chance to make AI players less predictable.
- The AI players need to be able to draw cards, and play cards
- Have dynamic aggression depending on if there winning or losing compared to the player next to them
- During the game there would need to be a clockwise and anticlockwise order, that can be switch if needed
- All normal playing cards and symbol/special cards need to be fully working and integrated into the game
- The game need a hub menu where it clicks way to start a game, view yours and other stats and quit the game
- The game needs win/draw/loss conditions that when triggered tell the user if they won/loss/drawn and update their stats
- The player needs to see all his cards in easy and fast way to be able to decide which card he wants to play with the uses of multiple pages to keep the screen less cluttered
- The player need to be able to play, pick up all types of cards if they have a move available
- The user needs to be punished for playing cards that are not playable compared with the top card. This would be done by the user being forced to pick up and not able to play the card
- The player needs to have the option when picking up to either play the card he picked up or play the card. However, a special condition needs to be made to void the player move/the card played if it can't be played compared to the top card, however, without punishment
- The game needs a huge amount of assets either made by me with the use of adobe photoshop or downloaded with a licence. The assets that are needed for the game are buttons, cards, fonts, backgrounds, etc

Design

About this section

Design section is a detailed description about the application. In the design section there is a detailed description comparing the GUI and terminal version of the application, and how the GUI works. The file structure of application, the use of decomposition and loggers to create flexible and maintainable code. Late additions to application including new sections. This sections also include information about algorithms included in the application, sql queries, the data structures implemented and the use of OOP in the application.

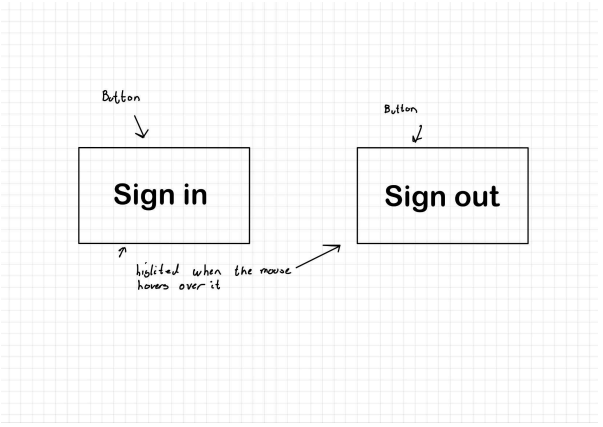
GUI vs Terminal

Graphical User Interface, more commonly known by its acronym GUI, are used to create in most cases an easier and more user-friendly experience. This is done with use of icons, buttons, menus and ect to allow the user to interact with the code. GUIs are a form of user interface (UI). The main difference between the two is that GUIs can present a user with a simpler overview of their choices with easier ways to interact with the program while hiding unnecessary information from the user. This can be seen in the majority of modern applications for example in word, a user has many buttons, menus and ect for quick changes from one click away without being taken out of the experience.

When I was creating a digital recreation of Uno, I created two versions of the application. One with a terminal and one with a graphical user interface. The terminal version of the application was created first so I knew how I wanted the program to work and the design was made on the fly. On the other hand, the graphical user interface had careful planning involved in design and execution. So in creating my GUI, I took three major steps: diagrams of the GUI, creating the logic/code and finally a fully working GUI.

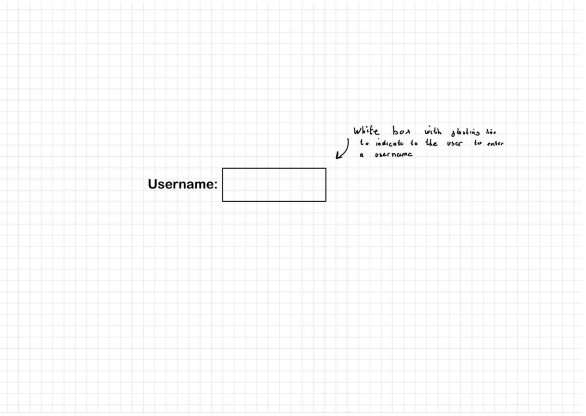
Login & Sign Up Menus

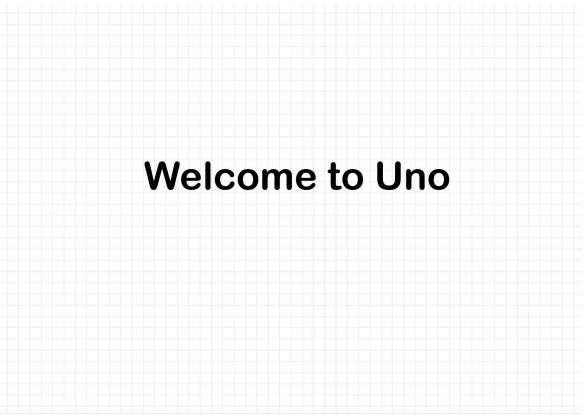
When the application is executed we should be shown two buttons with the option to sign in or sign out. These two buttons should display a brighter version of themselves if a mouse is hovering above them. When they are pressed we should be greeted by a new screen. (next diagram)



The point of this screen is to allow the user to type in either the username they would want to sign up/sign in with. It has text that says “username:” with a white box to indicatewriting space.

The box should not be clickable to start typing as we the program should assume they are going to type. One thing that is not here, but is there in the application is a back button in the top right. If the user is pressed after typing in a valid username it goes to the next screen (welcome uno diagram) else it goes back to the menu. If the user types in a valid username they are shown a welcome screen and after a bit there is taken to the game's main menu. One thing added later, the text would say “Hi (player name) welcome to uno”.





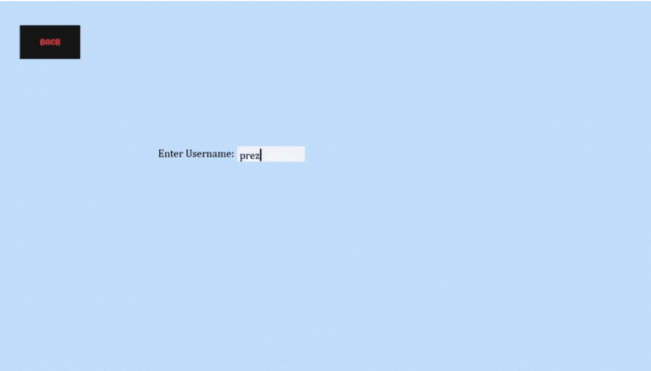
The terminal version of Uno. In this version, you just have to type all options. It would access a diagram to either check if you hadn’t made an account or if you have to depend on your choice. If an option you type in wasn't valid was met with the appropriate error code and booted back to the login/sign-up menu. Click the link to see the video: https://youtu.be/ZsGLVMZzpYc

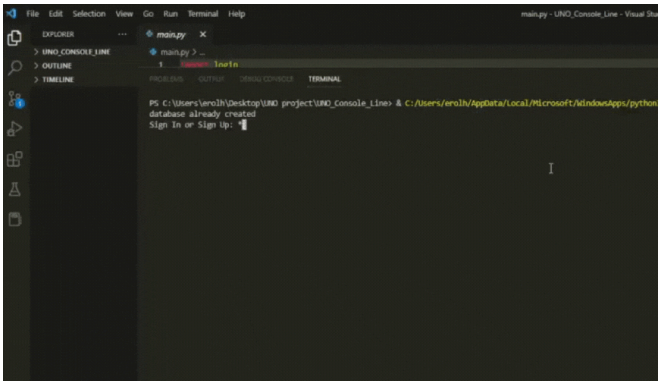
This is the GUIs version of the game. The versions follow the diagrams to create a fully working GUI. However, in this version, the limit of strings was changed from a max of ten to a max of five. Click the link to see the video: https://youtu.be/nUsi9dgE2VM

Error codes comparison GUI vs terminal

https://youtu.be/-mB3K1GjuQM

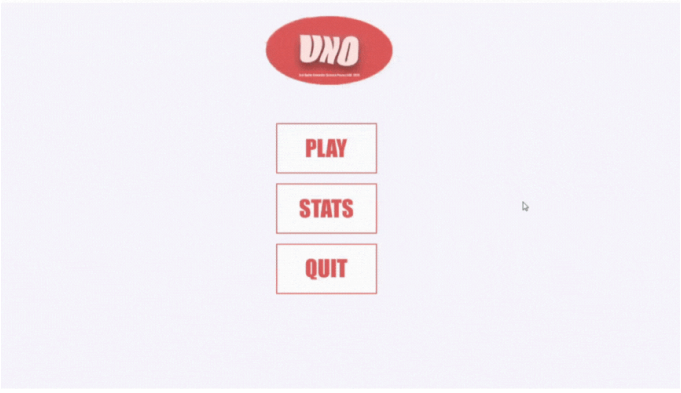
https://youtu.be/GbPx6oeRwVE

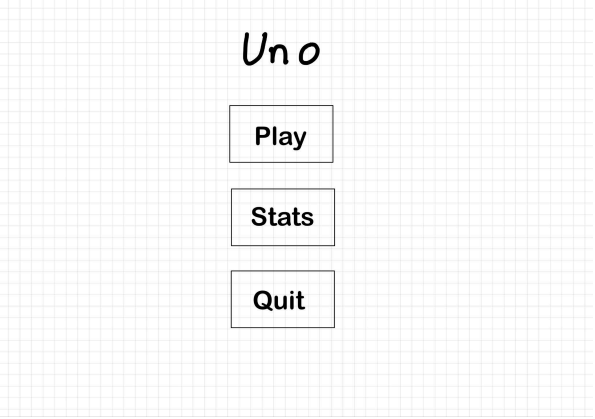




Main Menu

The original diagram/plan for the GUI shows a Menu screen with a UNO logo at the top of the screen with three buttons underneath it. The buttons would be play, stats and quit. The play button takes the user to the game while the stats button takes them to the leaderboard section. Quit button exit the program.
All buttons are highlighted when the mouse hovers over the buttons





The GUI version of the UNO main menu follows the diagram completely with all features mentioned before fully recreated in the application. Click the link to see the video: https://youtu.be/J3S9-RsQklA

The terminal version of the UNO main menu design is a bit different from the original diagram. The terminal uses text with “|” to separate options that you would type in. This menu also had an extra menu option called “settings”. This menu option was going to have me read a file that basically opened a text document and told the user the rules. However, when making the GUI I found the use of it not necessary and went back to the original three options. The terminal “setting” options can be seen below with placeholder text. Click the link to see the video: https://youtu.be/7VwWQA9otnM

```
Menu Option: | Rules | Menu |

Enter A Option: rules
.........
UNO ReadMe/Rules temp
The rules of UNO are simple and easy to explain. In fact, it's a great game for children and adults to play together (recommended age for kids is 7 years and up, but you could certainly try it with younger kids). Each ga
me takes only five to 10 minutes to play, so it's perfect for those with short attention spans or only a few minutes to spare â€" although some families have been known to play the game for hours!

UNO is played using a deck of 108 cards in four-color suits, numbered from 0 through 9: 19 blue, 19 green, 19 red and 19 yellow. The deck also includes eight "Skip" cards, eight "Draw Two" cards, eight "Reverse" cards, f
our "Wild" cards and four "Wild Draw Four" cards. A more recent version of UNO will include "Wild Swap Hands" and "Wild" customizable cards, two of each. Just remove them if you prefer classic UNO. Here's a brief rundown
 of the game's rules:

Object: The object of UNO is playing all the cards in your hand. When only one card is remaining, the cardholder must shout "Uno!" alerting others that he or she is down to the last card in his or her hand.

Choosing a dealer: A simple draw of cards determines the game's dealer; the person who chooses the card with the highest numerical value deals. If a player selects a non-numerical card, he or she returns it to the deck a
nd chooses another.

How to play: Each player is dealt seven cards at the game's start, with the remaining ones placed facedown to form a "draw" pile. The top card of the draw pile is turned over to begin a "discard" pile.

The first player has to match the card in the discard pile either by number, color or word. For example, if the card played is a red 7, the next player must throw down a red card or a 7 of any color. The player can also
play a Wild or a Wild Draw Four card, the latter only if he or she doesn't have a playable card in that color. If the player doesn't have a playable card, he or she must pick a card from the draw pile and play it, if pos
sible.

If the card plucked from the draw pile isn't playable, the next player must take his or her turn. When a player has one card left, he or she must yell "Uno!" If the player forgets, he or she has to pick two cards from th
e draw pile. So much for imminent victory.

Once a player has no cards left, the hand is over. Points are scored, and the dealer doles out new hands to everyone.

Scoring UNO is simple. When a player plays all his or her cards, the others count their points based on their remaining cards. Numbered cards are face value, colored special cards are worth 20 points and Wild cards are w
orth 50 points. The first player to play all of his or her cards receives points for the cards left in the opponents' hands. The first person to reach 500 points is declared the winner.

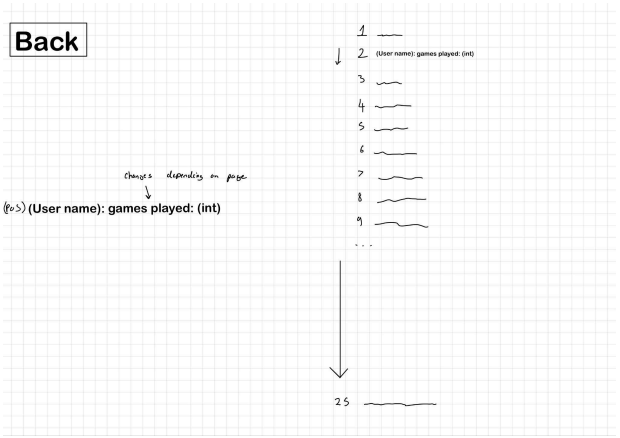
.........
```

Stats/Leaderboard

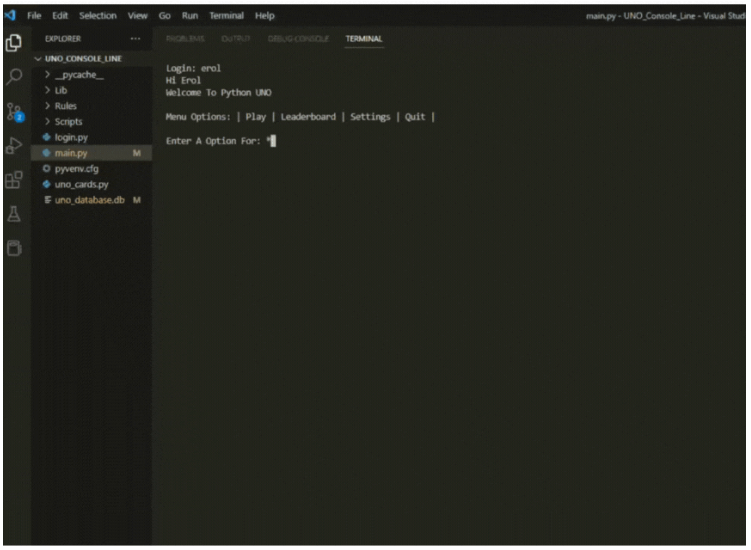


During the planning stage of my project where I was designing the GUI of the application. I had two versions of how to select what

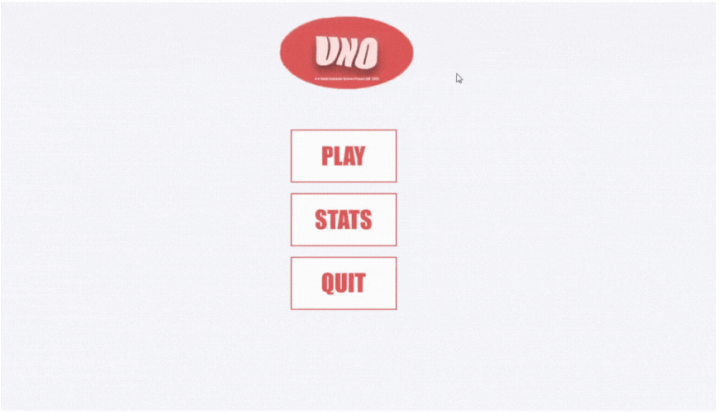
leaderboard the player wanted to see. One with intractable buttons that take the user to the leaderboard and to go back they would press the back button. However, during making the GUI I changed my mind and switched to the version as seen on the diagram. Instead of buttons, the user presses keys to quickly switch between leaderboard menus. The reason for the change was I wanted something different for the leaderboard and I found switching between menus faster.



In the terminal version of UNO the game leaderboard was relatively neat. It was made with a mix of SQL, Python and SQLite modules. The program would fetch all data from a user-selected choice and produce a list with only the necessary information. I figured out a little trick to maintain a nice row between the position of the user, the name of the user and games won/lost/ect. I would find the length of the string and take it away by number e.g 2,5 or 10. After I would print a formatted string with spaces times by distances to create the straight row appearances.



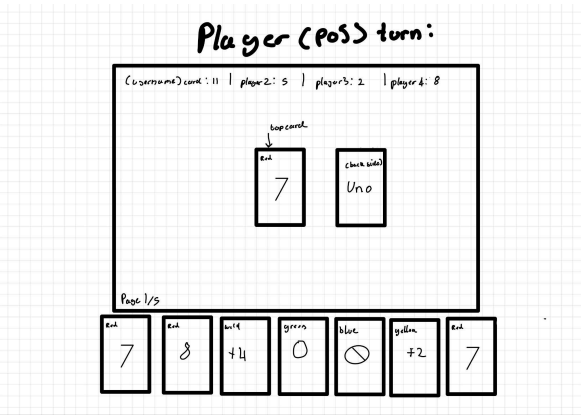
In the GUI version, I used the same code with slight changes to adapt to pygame. However in the example, as seen on the right the string when drawn would come up crooked. In this example, this problem is amplified by some strings in the database being longer than ten strings . This is because I've been using the same

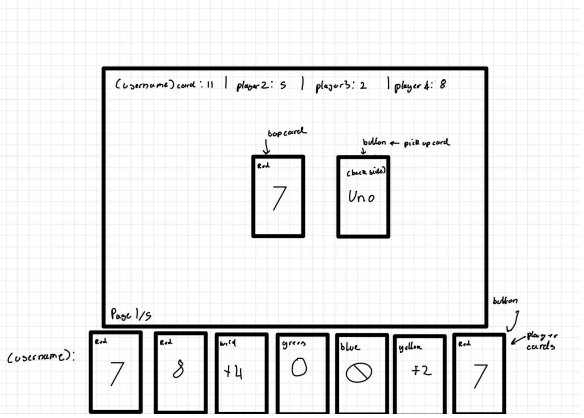


database since changing the string limit when signing up from ten to five. This is caused by pygame interpreters fonts and strings.

Game

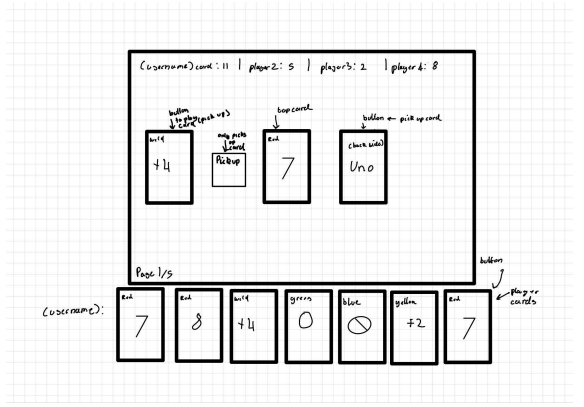
The plan for the game was to have a square table in the middle of the screen that was represented by a square border. On this table, there would be two cards. One card was to show the card on top and one would be a button that would give the user a new card. This is seen on the third diagram where there would be two options one that was a card that is a button that would play the card and a pickup option button would just pick the card up. At the top of the table there would be a string that displayed all the amount of cards other players have, as it would be information that players would have in the real game



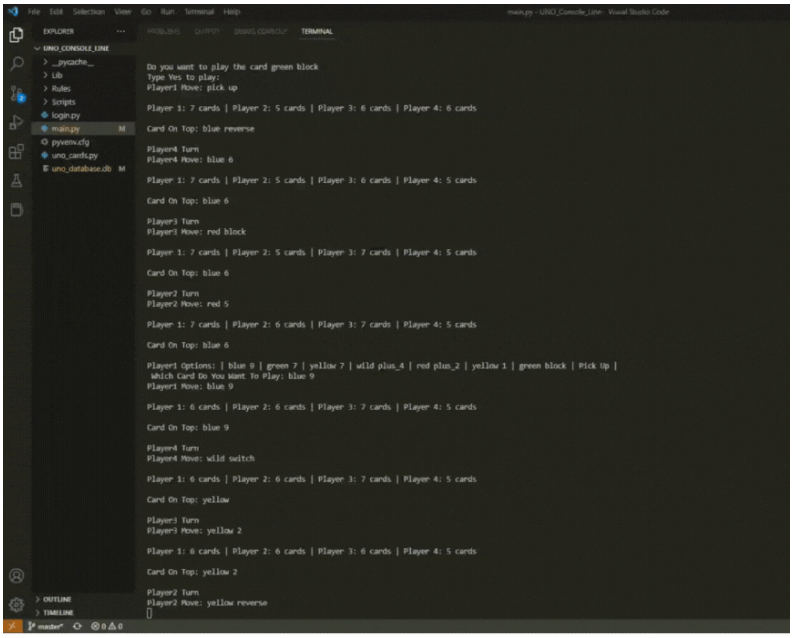


Underneath the table there are multiple pages of cards. Each page can have a max of 7 cards displayed at a time. Each card is a button that is pressed to play the card and replace the top card. With the use of keys, one to five, it would change the page to display the player's cards

In the first diagram there shows what happens when it wasn't the player's turn, and it was the AI player's. Above the table was the current player and all cards would turn into images only.



In the terminal version of the game, you would have to type in what card you wanted to play and if you type a string/card that wasn't in the deck it would give an extra card. However, this is not how you would pick up a card. To pick up a card you would have to type “pick up” and you were greeted with the new card and the option to play it or not. The AI players would play the card, you would see the top current card they would play and the card on the top. All the differences can be seen in the link: https://youtu.be/BC0EBJ3Stxo

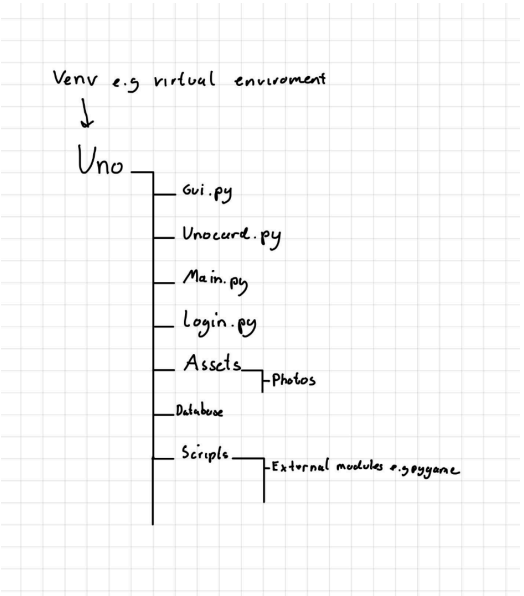


The GUI version follows the diagrams. The only difference is when a wild card is played the player is given four colour options that can be pressed to change the colour. Another change was the removal of the page string. A video of the GUI (in a future build) can be seen in the link:



https://www.youtube.com/watch?v=sZ9qxBPcGJw at 2:04

File Structure



The file structure shows where each file is in the directory. UNO is a folder that holds all the files in the program. The python files are gui.py, extra_module.py, unocard.py, main.py and login.py. The host file is the main file and is the only runnable file without being called by another file. The main file is where the main menu, leaderboard and a tiny bit of the UNO game are run from. A GUI file is a major component of the application as its sole purpose is to create a GUI. The file is called multiple times by all files, outside the extra_module.py. The login file is called at the start where it accesses a database and with the use of SQLite, it reads and writes information to and from the database. After it returns a string back to the main file, what it uses is to set the current player username. Within the main file, the user is given a condition on what function they want to do next. If they choose to play the game it calls the unocard.py file, where unocards.py receives information and sends it back to the main.py file, until a game condition is met. The GUI.py is the only file that has access to the Asset folder that holds the photos used for the GUI design. GUI.py loads these images and saves them as variables to be accessed later.

In the application the use of logger is used. A logger is a tool tha provides the ability to ttrace out errors or events of an application. A logger is great way to help montoiur and aduit applications by creating log meeasges with diffrentsds eleves, handlers, fromateers, and filters. In python I used the logging module to this, and I placed log messages around the code to be called to help in testing.

All files were created within the virtual environment to contain all external modules. Virtual environments is a way of isolating an environment and to have more control and flexibility over your project, so that it can have its own dependencies and packages independent of other projects. This can help avoid conflicts and compatibility issues when people download the application due to the third party libraries. Virtual environments can be used in a way of decomposition to break down a project into smaller components. The UNO folder directory was uploaded to a GitHub repository.

Late Additions

Further the development of the game moved the more ambitions I became to add more. These new ambitions were the addition of two new game modes, no wilds and numbers, a more detailed leaderboard section for the new game modes and a few tweaks to make navigation between menus more streamlined.

Gamemodes

As mentioned previously the addition of two new game modes: no wilds and numbers. The difference between the game modes is the type of deck that is used. In no wilds, the deck is removed of all wild cards e.g. plus 4. On the other hand, the numbers game mode is the removal of all special cards e.g. plus 4, plus 2, blocks and ect. These game modes retain the same design and the same way of winning and losing. However, with the addition of the two new game modes came the overhaul of the game database. For the new game modes each of them gets their individual tables where they retain the same attributes but their data will only be updated when they play their mode. For example, if the player wins a game in no wilds, the NoWilds table will have games played and wins fields would be updated, and not the other tables that share the same username. The play button, in the main menu, had to be rewritten to call a function to access the new menu, where the user would have a chance to choose three buttons to decide what they would want to play. A press of the button would ultimately decide on what deck is used in the game and would start the game..

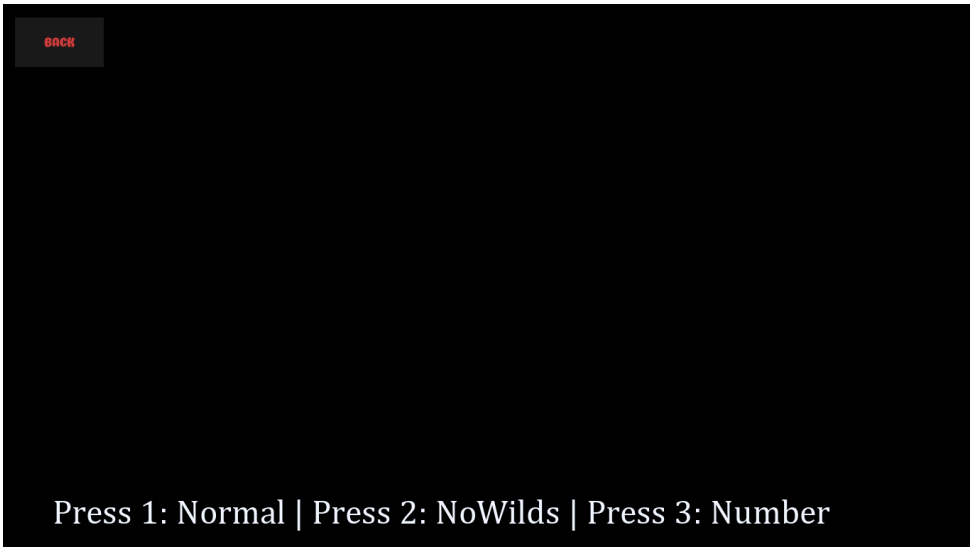


Leaderboard/Stats

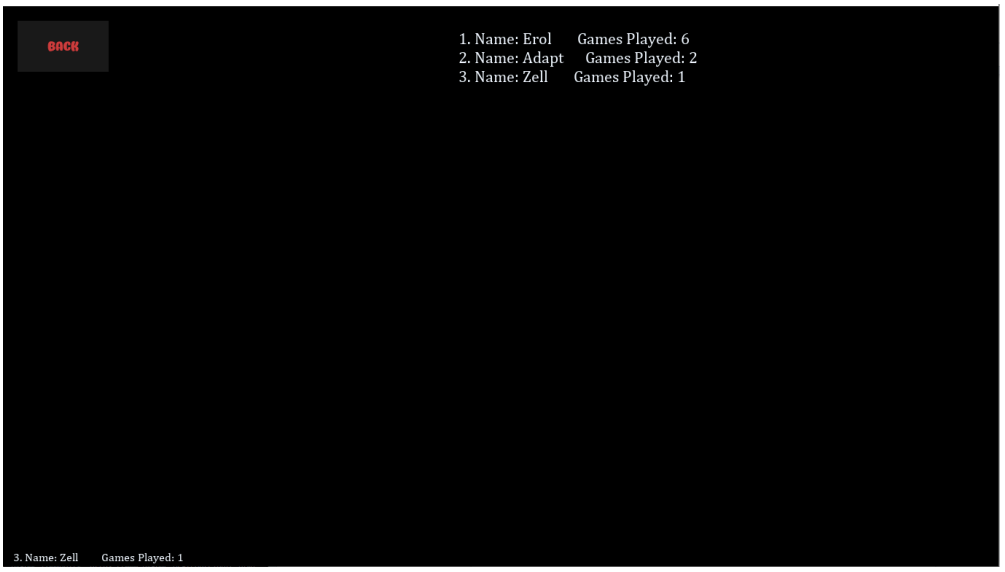
Due to the new tables added with the new modes. The leaderboard needed a major revamp. When the stats button, in the main menu, is pressed the user is redirected to a screen where they are given the option of two buttons, compare and separate.



If the separate button is pressed the user displays a text with all the user's possible next actions. Each of the actions (from the click of one of the number keys) decides what table data they would like to see. The user would then be shown a screen of the original leaderboard menu with the same options (wins, losses, games, draws), but with their selected table. The only difference to the original is the placement of the current user place placement. (screenshot shown in order)



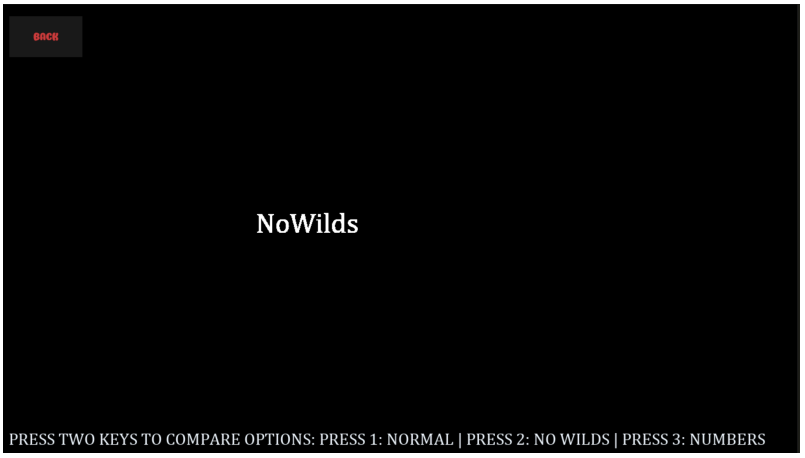


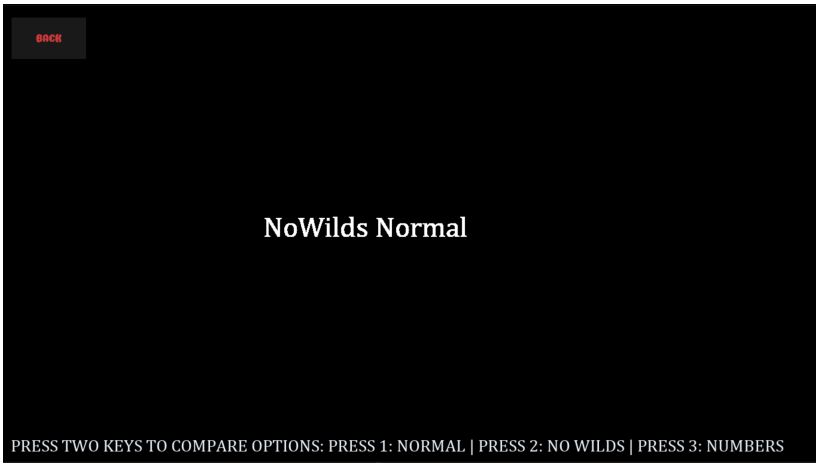


The compare button/options implementation allows the user to compare their stats from two different game modes. When a button is pressed the user is shown a new menu with the option of three options to create nine different comparisons. The options are chosen with the use of Numbered keys. The user can select two game modes that are displayed on the screen to show they have been selected with a third press to be redirected to the next menu. This menu gives options for games, wins, losses and draws, to select the options the user number keys. When an option is chosen the user is shown a leaderboard (in screenshots below the game's option chosen) with selected choices. The leaderboard shows the name of the position, the user, the first selected game mode, and what stat they want to see e.g games, the amount of stat the following repeats this but with the other game mode not including the user name plus the position. The leaderboard is ordered in descending order by the first select game mode stats. To check other states within the selected game modes all the user has to do is press a different number key. To select different modes, the user just needs to press the back button and choose their new options. (screenshots in order, plus the word “ganes” has now been replaced with “games”

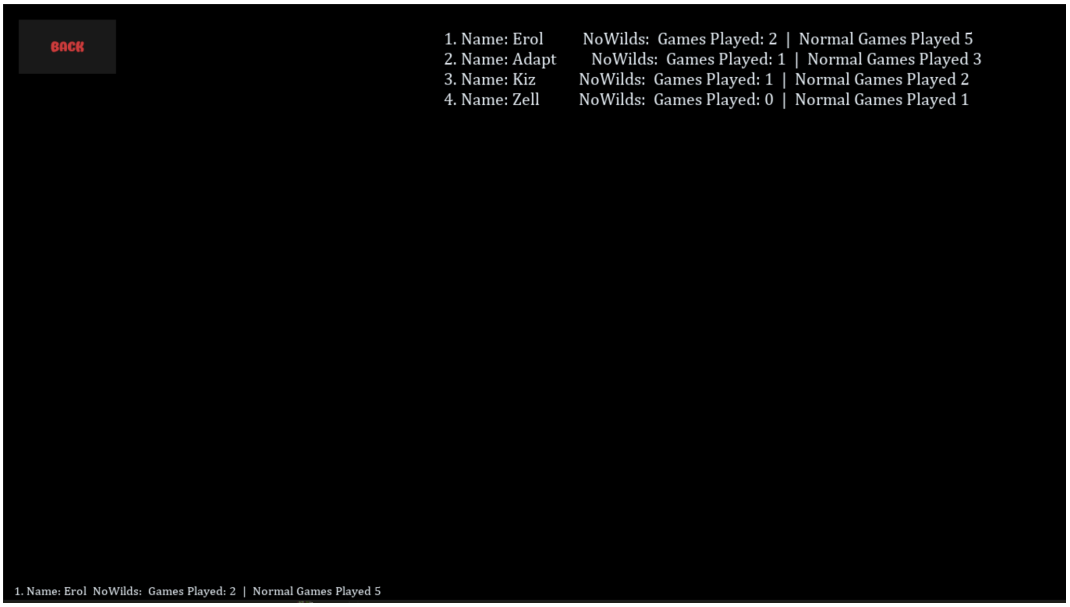
as these screenshots are from an old build). Link to see the menu in action: https://youtu.be/_HLId5m2p8M









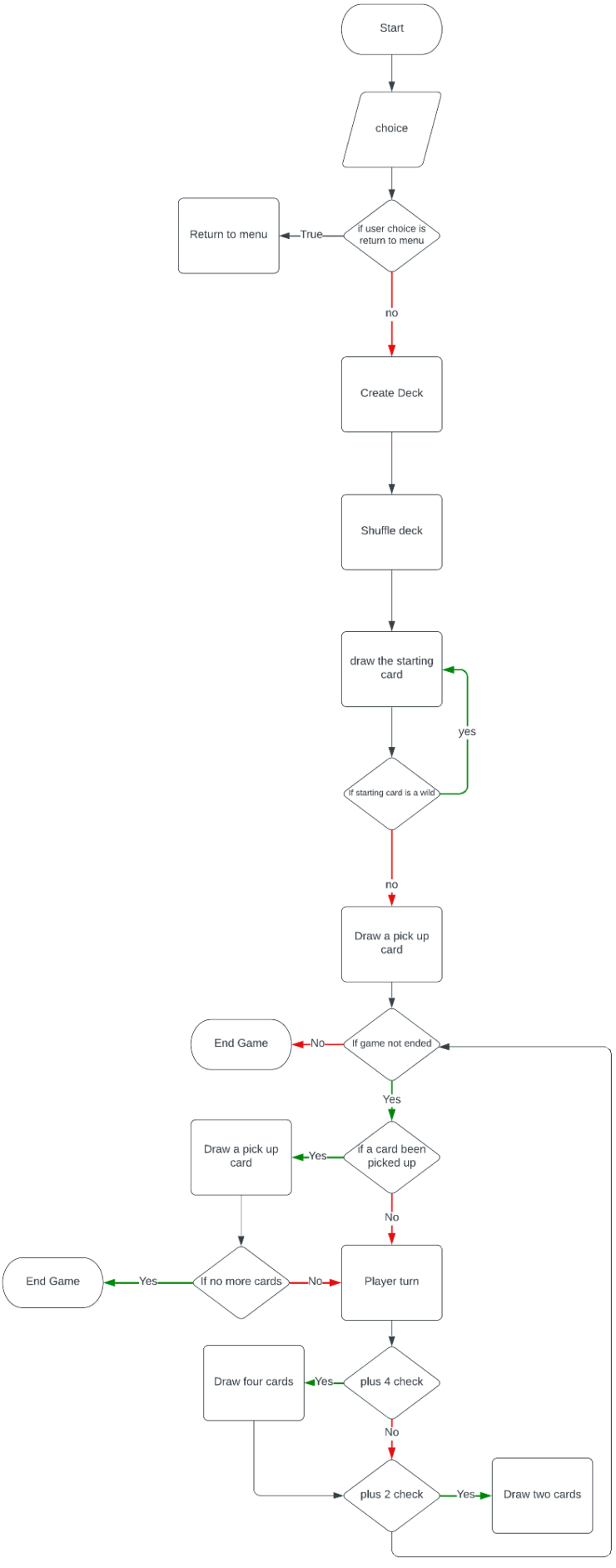


Minor Tweaks

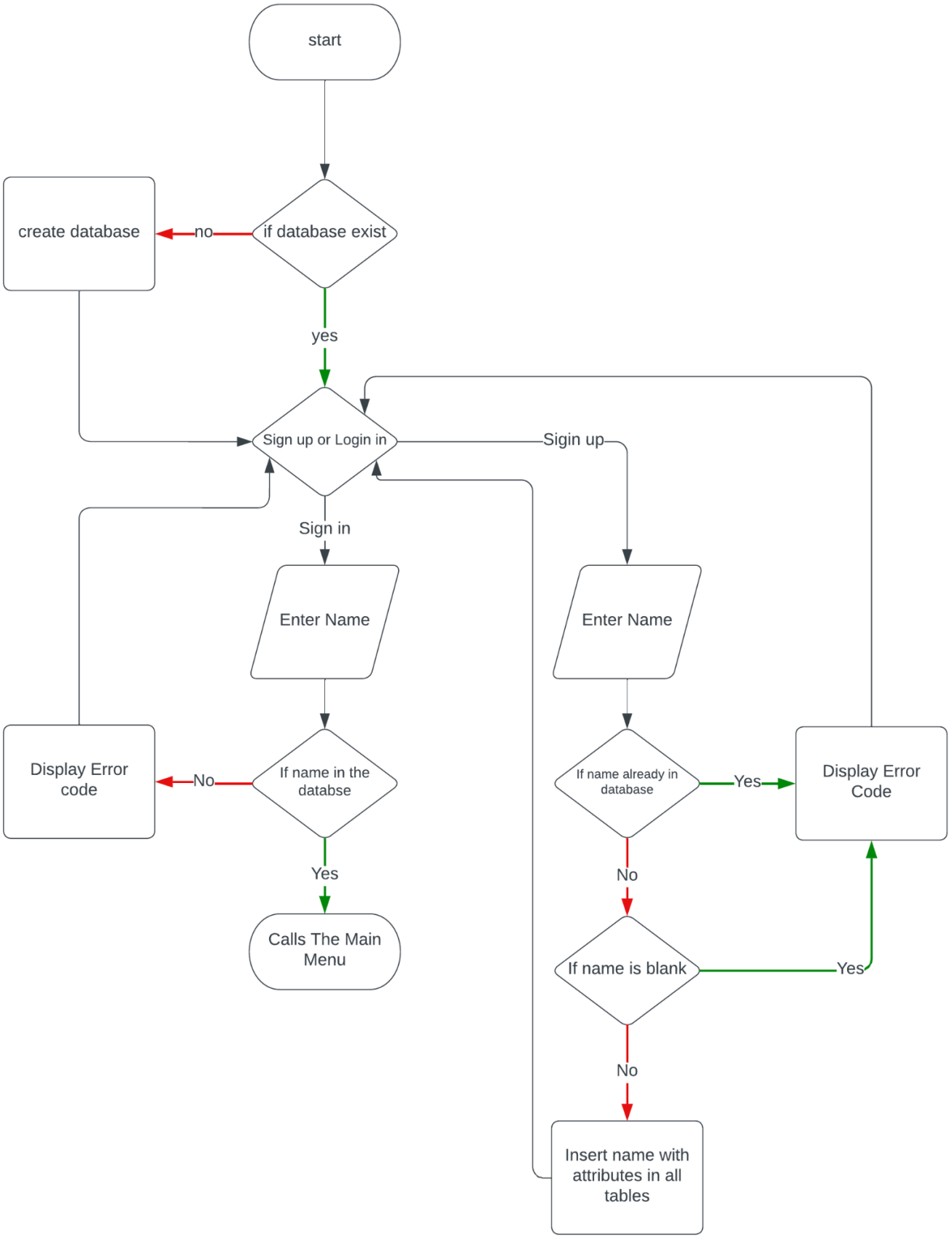
- ➢ The addition of a back button in the main menu to return the user to log in or sign up. This allows the user to change or create a new user to play without exiting the application
- ➢ The background on the main menu changes to a white background
- ➢ Fixed spelling of games in the leaderboard menu
- ➢ Draw screen lasts a shorter period

Flow Charts

Main game function flow chart



Login Menu meun function flow chart



Code / Data Structures / Algorithms / OOP

Data Structures

In computer science, a data structure is a data organisation, management, and storage format that is usually chosen for efficient access to data. Data structures are usually used for accessing and manipulating data with the use of pointers e.g. queues, lists, vectors and ect. Data structures can be dynamic or static. Dynamic data structures refer to a collection of data in memory that can grow or shrink, with the use of a heap. Heap is the portion of memory in which space is automatically allocated or deallocated. However, excessive allocation of memory without deallocation may cause overflow. On the other hand, static data structures cannot grow or free space, as memory cannot be reallocated during execution. In python, there are only four built-in data structures: Lists, Tuples, Sets and Dictionaries. However, we can recreate data structures that are not built in with other data structures e.g, in python using a list and pointers to create a queue. Abstract Data Type is a data type, where only behaviour is defined but not its implementation. Abstract data types allow us to view data and perform operations that are allowed without regard to whether they will be implemented. E.g in a stack the data types would be peek, pop, push and ect. In the application, the data structures used were: Lists and Dictionaries.

List/ Linked list work is a dynamic data structure. Lists are ordered, and use pointers dictate this order. So if we had several items already in the list, each would point to the next one in order, if the user wanted to add an item to the end of the list they would use ADT (append). What would happen is the data being added would have a pointer that would point to the last item in the list. If an item is removed the pointers or pointer-connected items are either removed or redirected to the next item depending on the position of the data. To insert data a pointer would be directed to the next item, and the previous item pointer would be redirected to the inserted item. Lists in python are built in. We can do several things to create unique things.

An example of an ordinary list in python is:

```
self.__colour = ["red","green","blue","yellow"]
```

We can also have a nested list, a list within a list:

```
amount_of_card_weight = [["red",red_count],["green",green_count],["yellow",yellow_count],["blue",blue_count]]
```

We can use list comprehension to create a list with many items, without having to type each individual variable, string, integer and ect. The list can have operations in python to duplicate or add data.

```
self.__value = [str(i) for i in (range(0,10))]*2
```

We can also combine this with composition, to create several objects and have them saved in a list.

```
card = [cards(c,v) for c in self.__colour for v in self.__value] + [cards(c,v) for c in self.__colour for v in self.__special]*2
```

We can use lambda in python to create small functions without any name to create lists:

```
list_for_cards = list(filter(lambda x: x[0] == top_current_card[0] or x[1] == top_current_card[1] or x[0] == "wild",list_for_cards)) + [["pick_up",None,0]]
```

We can create a nested list with list comprehension:

```
user_cards = [[pygame.image.load(f'Assets/Cards/{i}.png').convert_alpha(),str(i)] for i in user_deck]
```

To access values in the list we find the index of where the item we want to extract from the list. We either use the index operation or add square brackets after a variable with the index to access it (if it is nested add another pair of brackets to access it). In the example below self._order is a nested list within, each list in multiple objects. To access the list within the list we use the variable self._order_postion as a pointer that increments or decrements in a circle. This gives the current position of what data we want to access.

```
def next_turn(self):
    """Changes who turn it is to play. Check which way the order is going too"""
    if self._clockwise == True:
        if self._order_postion == 3:
            self.set_order_clockwise()
        else:
            self._order_postion = self._order_postion + 1
    elif self._clockwise == False:
        if self._order_postion == 0:
            self.set_order_anti_clockwise()
        else:
            self._order_postion = self._order_postion - 1

def card_in_deck(self,pick_up_card):
    """Check if the card trying to be played is in the deck. Returns True if the if it in the deck else it returns False
    and force the user pick up a card without being able to play"""

    for i in self._order[self._order_postion]:
        if str(i) == str(self._player_move):
            return True

    if self._pickup == False:
        self._order[self._order_postion].append(pick_up_card)
    return False
```

This is only a brief showing of how the list data structure has been used in the project to manipulate and hold data.

The other data structure used in the project was a static dictionary. The dictionary is an abstract data type consisting of pairs, each having a key and a value. Dictionaries are similar to hash tables but have a few differences. Hash tables use a hash function to compute the index of an item in the array, whereas dictionaries may use different algorithms to determine the location of an item in the data structure hash tables are typically faster for lookups and insertions than dictionaries, as the hash function allows for constant time complexity O(1). In python, they currently implement their built-in dictionaries using hash tables. The reason for the use of data structure in the project is Fast lookups, Key-value storage, and Flexibility (can store a multitude of data structures).

In the application, we have one dictionary that holds the weights of the card's abilities. The higher the amount the more powerful the card is. This dictionary is made for the AI, and its use is to help the AI to weigh up its options and choose the best cards to play. The key is the card name and the value is the weight of the card. This dictionary is static and does not grow or shrink in max size and keys and values are not altered.

```
self._cards_weight = {"plus_2":5,"plus_4":10,"block":3,"reverse":2,"switch":5}
```

The code below shows the use of the implementation of a dictionary. The code starts by accessing the current AI player deck, and iterating through all the cards, making them one by one equaling “i”. “i” is then split up to distinguish what type of card it is and does have a special ability e.g. plus 4, plus 2 or skip go/block and ect. The variable “i” is potentially equal to any key in the dictionary, it receives its weight/value or it is given a weight of 1. This is then saved to a variable called “g”. The card colour, value and weight are put in a list where they are appended to another list. The code and how it works can be seen below.

```
for i in self._order[self._order_postion]:
    i = str(i).split() #splits cards into two strings e.g ["red" "2"]
    g = self._cards_weight.get(i[1],1) #g uses i to get the cards values/weights from a dictionay
    print(f"This is g: {g}")
    print(f"This after: {[i[0],i[1],g]}")
    list_for_cards.append([i[0],i[1],g]) #appends to a list with the: card and its weights
```

```
This is g: 1
This after: ['red', '4', 1]
This is g: 1
This after: ['green', '3', 1]
This is g: 1
This after: ['blue', '3', 1]
This is g: 1
This after: ['yellow', '0', 1]
This is g: 10
This after: ['wild', 'plus_4', 10]
```

Algorithms

A sorting algorithm is an algorithm that arranges the elements of a list or array in a particular order. There are many different sorting algorithms, each with its own strengths and weaknesses. Time complexity is used to measure the running time of an algorithm as the input size increases. This is often expressed in Big O notation, a quick notation of how fast the time of running increases as the size of the input increases e.g O(n^2) is the time complexity of bubble sort as it iterates over twice to sort data, making it very inefficient for large data. Space complexity measures the amount of memory an algorithm needs to run as the size of the input increases. It is also expressed in Big O notation, which shows the total amount of memory space used by an algorithm as a notation e.g. recursive merge sort has a space complexity of O(n) in the worst case, as it may require additional memory equal to the size of the input array to store the subarrays during the sorting process. The application uses two different sorting algorithms: recursive merge sort and quicksort, both held in the extra_module.py file. Merge sort and quicksort are both divide-and-conquer sorting algorithms that have different time and space complexity.

Merge sort has a time complexity of O(n log n) in all cases where n is the size of the input array and as stated beforehand it has a space complexity of O(n). When an algorithm is recursive, it keeps calling itself until it is instructed to stop, as if doing so continually might cause overflow. Recursive merge sort works by

1. Splitting the array in half. If the array contains an odd number of elements, one half will include one more element than the other.
2. Each half is recursively split in half and the merge sort algorithm is called on each half. As a subarray of size one is already sorted, this process is repeated until each subarray has just one element.
3. We must combine the sorted arrays. To merge two sorted subarrays, we compare the first element of each subarray and choose the smaller one to be inserted in a new array first. We then repeat this process until all of the elements from both subarrays are included in the new sorted array.
4. This process is repeated for each pair of subarrays until the entire array is sorted.

The merge sort algorithm is one of the most efficient sorting algorithms, particularly for huge arrays. The algorithm's code is shown below; it has been modified to only sort nested lists.

```python
def merge_sort(list1):
    if len(list1) == 1:
        return list1
    left_list = list1[:len(list1)//2]
    right_list = list1[len(list1)//2:]
    left_list = merge_sort(left_list)
    right_list = merge_sort(right_list)
    results = []
    while len(left_list) > 0 and len(right_list) > 0:
        if left_list[0][1] > right_list[0][1]:
            results.append(left_list.pop(0))
        else:
            results.append(right_list.pop(0))
    results.extend(left_list)
    results.extend(right_list)
    return results

```

```
608                 amount_of_card_weight = extra_module.merge_sort(amount_of_card_weight)
```

QuickSort, like Merge Sort, is a Divide and Conquer algorithm. Quick sort is an algorithm which divides a list into two smaller sub-lists and recursively sorts each sub-list. It has a time complexity of O(n log n) for a list of n elements. However, the worst case can be O(n^2). Quick sort works by

1. Select a pivot: Pick an element from the list to serve as the pivot. The pivot is typically the first or last item on the list. In this implementation, it is the first item.
2. Partition: Rearrange the list so that all elements less than the pivot come before it, and all elements greater than the pivot come after it.
3. Recursion: The list is now divided into three sub-lists: one with all elements less than the pivot, one with all elements greater than the pivot, and one with all elements equal to the pivot. Recursively apply QuickSort to all sub-lists recursively until the sub-list has zero or one element
4. Return: Merge the three sub-list together and return the final result

```
def quicksort(array):
  less = []
  equal = []
  greater = []

  if len(array) > 1:
    pivot = array[0]
    for i in array:
      if i[2] > pivot[2]:
        less.append(i)
      elif i[2] == pivot[2]:
        equal.append(i)
      elif i[2] < pivot[2]:
        greater.append(i)
    return quicksort(less) + equal + quicksort(greater)
  return array
```

```
206          list_for_cards = extra_module.quicksort(list_for_cards)
```

Quicksort is a highly efficient algorithm that is widely used in a wide range of programming languages and applications, however not as good as merge sort as its worst-case is more severe than merge sort.

Databases and SQL

Databases are collections of tables that store data about entities and use attributes to describe the data stored within the table. Each entity needs an identifier that identifies a record. In relational databases, data is held in tables, known as relations and these tables are linked due to a common attribute. This common attribute would be a primary key in its table and foreign in another. These keys are identifiers to link multiple data between different tables within a database. SQL is a declarative programming language, where you write statements that describe a problem to be solved, and the language decides the best way of doing it. SQL is a language made for the manipulation, creation and relation of databases. Python has a built-in module called SQLite which makes combining sql and python relatively simple.

When the application is run, with the built-in module, it checks to see if a file called ("uno_database.db") exists. If it is there it returns a boolean expression. If these expressions are False it calls the create function in the login module that creates the file, after it starts creating all the tables ("NoWilds", "Numbers", "Normal") with all their attributes of (Games INT, Wins INT, Loss INT, Draws INT, PRIMARY KEY('UserName')).

```
def create():
    conn = sqlite3.connect("uno_database.db")
    c = conn.cursor()
    for table_name in ["NoWilds","Numbers","Normal"]:
        c.execute(f"""CREATE TABLE {table_name} (
            UserName VARCHAR(10),
            Games INT,
            Wins INT,
            Loss INT,
            Draws INT,
            PRIMARY KEY('UserName'))""")
        conn.commit()
    conn.close()
```

After this or if that previous boolean expression was True, the variable user = to function in the GUI module called start loop, which uses buttons that either let users type in a username and if they want to sign in or sign up. It saves both of these to variables and calls a function with both variables in the login module called login_or_sign with parameters of login_options and name. This then creates an object with a User class. If the login option is to sign up, it calls a method called set_username. It checks within the Normal table for the selected username and if it already exists. If it is in a table or if the user name is blank it returns the user to the menu (startloop) with an error code. Else the username is inserted into all tables with all the other attributes set to zero. The user then returns back to the start loop.

If the user chose the sign-in option in the start loop, where the sign-in method was used. The method is used to try and accept conditional logic. So if the user is in the table no error is raised and the username is returned all the way back to the main module where it would be equal to the user variable. But if not in the table it raises a type error that returns the user with an error code back to the start loop. This seen on fig 0.1

```
def set_username(self,u):
    conn = sqlite3.connect("uno_database.db")
    c = conn.cursor()
    self.__username = u.replace(" ", "_")

    try:
        copy_check = c.execute(f'SELECT UserName FROM Normal WHERE UserName IS "{self.__username}"')
        copy_check = str("".join(c.fetchone()))
    except TypeError: None

    if copy_check == self.__username:
        self.error = "Please Create A UserName That Not In The Database"
    elif self.__username == "":
        self.error = "Please Do Not Leave The Username Blank"
    else:
        for table_name in ["NoWilds","Numbers","Normal"]:
            c.execute(f"INSERT INTO {table_name}(UserName,Games,Wins,Loss,Draws) VALUES ('{self.__username.lower()}',0,0,0,0)")

    self.__username = ""
    conn.commit()
    conn.close()
```

Fig 0.1

If the user chose the sign-in option in the start loop, where the sign-in method was used. The method is used to try and accept conditional logic. So if the user is in the table no error is raised and the username is returned all the way back to the main module where it would be equal to the user variable. But if not in the table it raises a type error that returns the user with an error code back to the start loop. This seen in the code below

```
    def sign_in(self,u):
        conn = sqlite3.connect("uno_database.db")
        c = conn.cursor()

        self.__username = str("".join([i for i in u])).lower()
        check = c.execute(f'SELECT UserName FROM Normal WHERE UserName IS "{self.__username}"')

        try:
            check = str("".join(c.fetchone())).lower()
        except TypeError:
            self.__username = ""
            self.error = "Please Enter A Valid Username"
            conn.close()

        if check == self.__username:
            return self.__username

    def get_user(self):
        return self.__username

    def error_code(self):
        display = self.error
        self.error = ""
        return display


def login_or_sign(login_options,name):
    account_name = User()
    if login_options == "sign up":
            account_name.set_username(name)

    elif login_options == "sign in":
        account_name.sign_in(name)

    list2 = [account_name.get_user(), account_name.error_code()]
    return list2
```

The next occurrence of SQL is in the uno_cards module and that main module. Both are used to update the user games, wins and losses in their game mode individual tables. In the method below we check all player's decks to see if anyone has zero cards in their decks. G variable is the number of cards and E is which player has zero cards. If g = 0, then the end variable = True. If the end is true, the player games are updated and incremented by one for the user. If g= 0 meaning the user had zero cards we call database_updatae that flexible function that finds specific data received from the parameters and increments it by one then saves it to the table. So if the user won the function updates the user wins otherwise if the user loses the function updates the user's losses.

```
def database_updates(user,worl):
    user_worl = c.execute(f"""SELECT Games, {worl}
    FROM {self._gamemode}
    WHERE UserName = '{user}'
    """)

    user_worl = [int(i) for i in c.fetchone()]
    logger.info(f"{user} Games Played {user_worl[0]} and {worl} {user_worl[1]}")
    user_worl[0] += 1
    user_worl[1] += 1
    logger.info(f"After it been incremented {user} Games Played {user_worl[0]} and {worl} {user_worl[1]}")

    c.execute(f"""UPDATE {self._gamemode}
    SET Games = {user_worl[0]}, {worl} = {user_worl[1]}
    WHERE UserName = '{user}'""")
```

The draw system in the game works very similarly from sql point of view, however, to trigger it is different. A draw occurs either if there are zero cards or the user back out of the game. So if one of these events happens we raise TypeError or IndexError to start the draw system.

```
except IndexError: #Game ended as draw

    logger.info("Game ended with a draw")
    gui.drawloop()
    con = sqlite3.connect("uno_database.db")
    c = con.cursor()

    table = c.execute(f"""SELECT Games,Draws
                FROM {choice}
                WHERE UserName = '{user}'
                """)
    table = [int(i) for i in c.fetchone()]
    logger.info(f"Games = {table[0]}, Draws = {table[1]}")
    table[0] += 1 # games
    table[1] += 1 # draws

    c.execute(f"""UPDATE {choice}
    SET Games = {table[0]}, Draws = {table[1]}
    WHERE UserName = '{user}'""")
    logger.info(f"Games played {table[0]}, User Draws {table[1]}")
    con.commit()
    con.close()
    break
```

SQL in the application can also be found in the leaderboard in the GUI module in the lead_menu function. In the first picture below, when a user wants to view a single stat from one table it will use the simple select SQL statement below to display the user's position, usernames, and the user choice (plus the user stats on the left of the screen).

```
def leaderboard_choice(choice,text,user,gamemode):
    """To access SQL Table and pull it infomation for it to be displayed"""

    con = sqlite3.connect("uno_database.db")
    c = con.cursor()

    c.execute(f"""SELECT UserName,{choice}
        From {gamemode}
        ORDER BY {choice} DESC
        """)
    board = []
    player_postions = [i for i in c.fetchall()]
    for i in range(0,len(player_postions)):
            if i < 99:
                username_length = 10 - len(player_postions[i][0])
                num_length = 2 - len(str(i+1))
                leader = (f"{' '*num_length}{i+1}. Name: {player_postions[i][0].capitalize() + ' '*username_length} {text}: {player_postions[i][1]}")
                board.append(leader)
                if player_postions[i][0] == user:
                    user_postion = i + 1
                    user_games = player_postions[i][1]
    username_length = 10 - len(user)
    num_length = 2 - len(str(user_postion))
    curent_user = f"{' '*num_length}{user_postion}. Name: {user.capitalize() + ' '*username_length}  {text}: {user_games}"
    return [board,curent_user]

page = 0
des.create()
while separte_continue == True:
    if page == 1:
        choice = leaderboard_choice("Games","Games Played",user,gamemode)
        contin = True

    elif page == 2:
        choice = leaderboard_choice("Wins","Ganes Won",user,gamemode)
        contin = True

    elif page == 3:
        choice = leaderboard_choice("Loss","Games Loss",user,gamemode)
        contin = True

    elif page == 4:
        choice = leaderboard_choice("Draws","Games Drawn",user,gamemode)
        contin = True
```

In the compare menu we use joins to compare relations. So the compare menu is a leaderboard that compares the user's stats between relations. In the application, we only compare two due to it being easier for users to compare stats without overwhelming them. If we were going to compare all of them we would just add another INNER JOIN to find where usernames (identifiers) are the same in other tables. Code on the other page

```
def leaderboard_choice(choice,text,user,gamemode1,gamemode2):
    """To access SQL Table and pull it infomation for it to be displayed"""

    con = sqlite3.connect("uno_database.db")
    c = con.cursor()

    c.execute(f"""SELECT (UserName,{choice})
        FROM {gamemode1} INNERJOIN {gamemode2}.UserName
        {gamemode1}.UserName == {gamemode2}.UserName
        ORDER BY {choice} DESC
        """)
    board = []
    player_postions = [i for i in c.fetchall()]
    print(player_postions)
    for i in range(0,len(player_postions)):
            if i < 99:
                username_length = 10 - len(player_postions[i][0])
                num_length = 2 - len(str(i+1))
                leader = (f"{' '*num_length}{i+1}. Name: {player_postions[i][0].capitalize() + ' '*username_length} {text}: {player_postions[i][1]}")
                board.append(leader)
                if player_postions[i][0] == user:
                    user_postion = i + 1
                    user_games = player_postions[i][1]
    username_length = 10 - len(user)
    num_length = 2 - len(str(user_postion))
    curent_user = f"{' '*num_length}{user_postion}. Name: {user.capitalize() + ' '*username_length}  {text}: {user_games}"
    return [board,curent_user]

page = 0
des.create()

while compaere_menu == True:
    if page == 1:

        choice = leaderboard_choice("Games","Games Played",user,optionslist[0],optionslist[1])
        contin = True

    elif page == 2:
        choice = leaderboard_choice("Wins","Ganes Won",user,optionslist[0],optionslist[1])
        contin = True

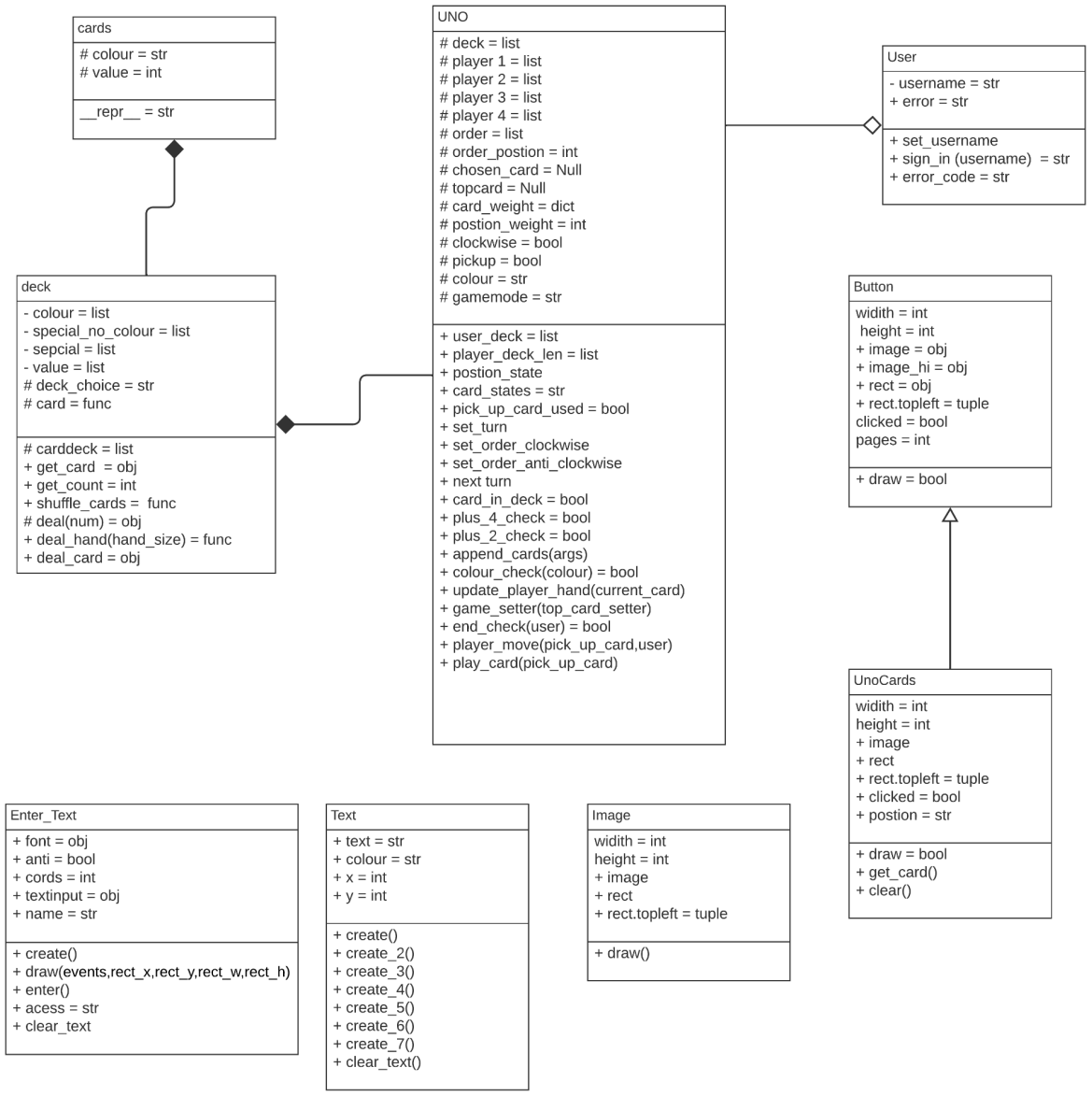
    elif page == 3:
        choice = leaderboard_choice("Loss","Games Loss",user,optionslist[0],optionslist[1])
        contin = True

    elif page == 4:
        choice = leaderboard_choice("Draws","Games Drawn",user,optionslist[0],optionslist[1])
        contin = True
```

Object Oriented Programming (OOP)

UML

- Private, # Protected + Public



Object-oriented programming is a programming paradigm supported by languages like Python and Java. OOP was created to make it possible to abstract details of its implementation away from the user, make code reusable, make maintenance of the code easier and is a good framework to use when creating libraries. OOP is a collection of

objects, these objects can be a place, an animal or in this case a card. In OOP each object has its own data (attributes) and operations (methods), objects that can interact with each other and all the processing is done by the objects. A class is a template/blueprint for an object and defines the attributes and methods of the objects in the class. A constructor is used to create objects based on the class.

Classes in OOP have access modifiers that dictate how hiding an attribute or method is from a user. These access modifiers are Public, Protected, and Private.

Access	Public	Protected	Private
Defining a class	Yes	Yes	Yes
Via Inheritance	Yes	Yes	No
Objects in a class	Yes	If it a subclass	No

In the application a mix of modifiers are used, mostly protected use. In application, private access modifiers are rarely used and in most cases, there wasn't a reason to completely hide information.

Communication between objects there are ways of approaching it, either using inheritance that has “is a “ relationship with objects and can be seen with a line with an arrow. It is used to transfer methods and data. On the other hand composition and aggregation, both have “has a” relationship. In the application, the use of aggregation is used. Aggregation is where the object remains when the container object is destroyed. The composition can be seen by an unfilled diamond and a solid line. The application uses a mix of aggregation, composition so other classes inherit data and inheritance to transfer methods to avoid duplicated code.

The reason for inheritance is to allow another class to inhertiate methods and attributes from another class. This is done with button and uno cards classes, as UnoCards inheritance. As UnoCards class is adaptions of the button class but only needs diffents but needs few diffrent methods and attributes for it behaviour. The reason for aggregation is so that the object lifetime is only finished when the object remnoved. This can be seen between the deck and UNO classes, as when new game created the previous object is removed and new one created removing deck objects within UNO. Composition is used for

the username, as we want the user name to life cycle to independent from other classes, as seen between the relasiobnship between Uno and User classes.

Technical Solution

Indentation may be out of place if so you can use this drive to view the files:
https://drive.google.com/drive/folders/1itOsZJJLVacBiPhT8JRBJ-Z02b_Nj5cB?usp=sharing

Main.py

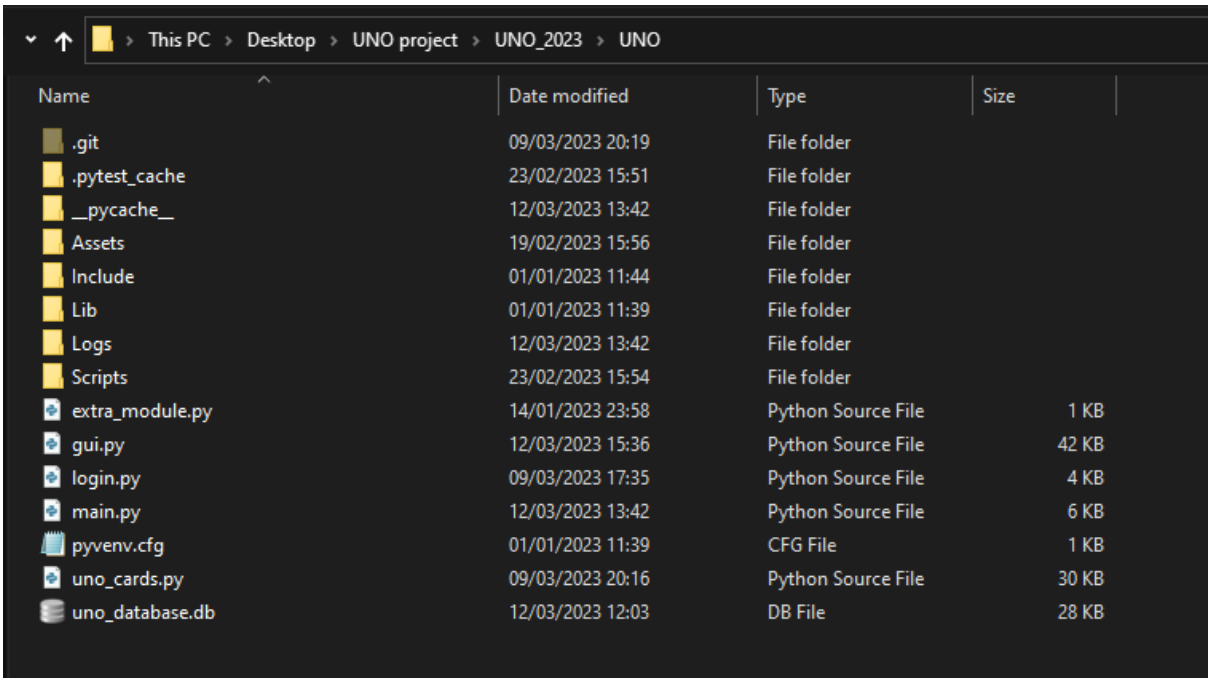
Login.py

Uno_cards.py

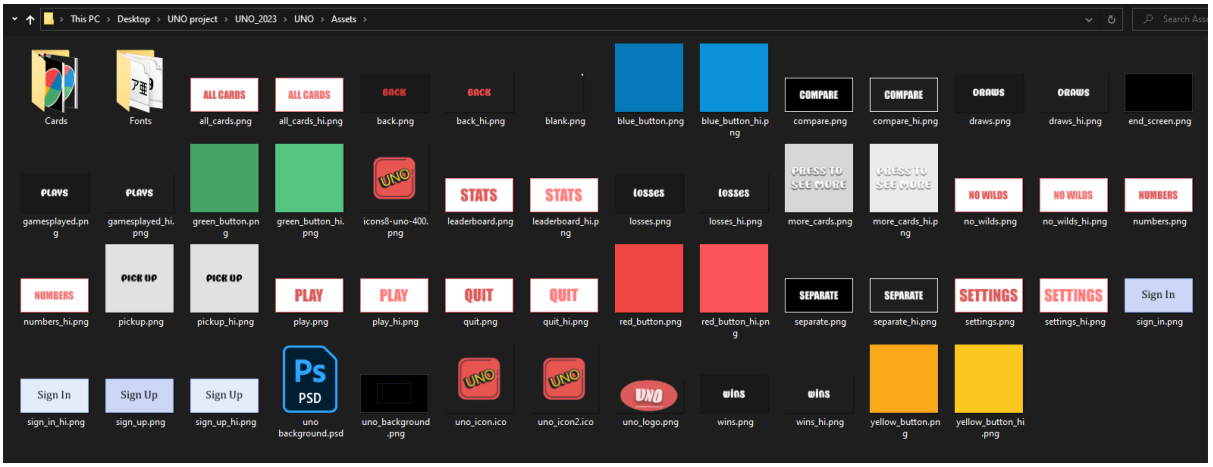
extra_module.py

Gui.py

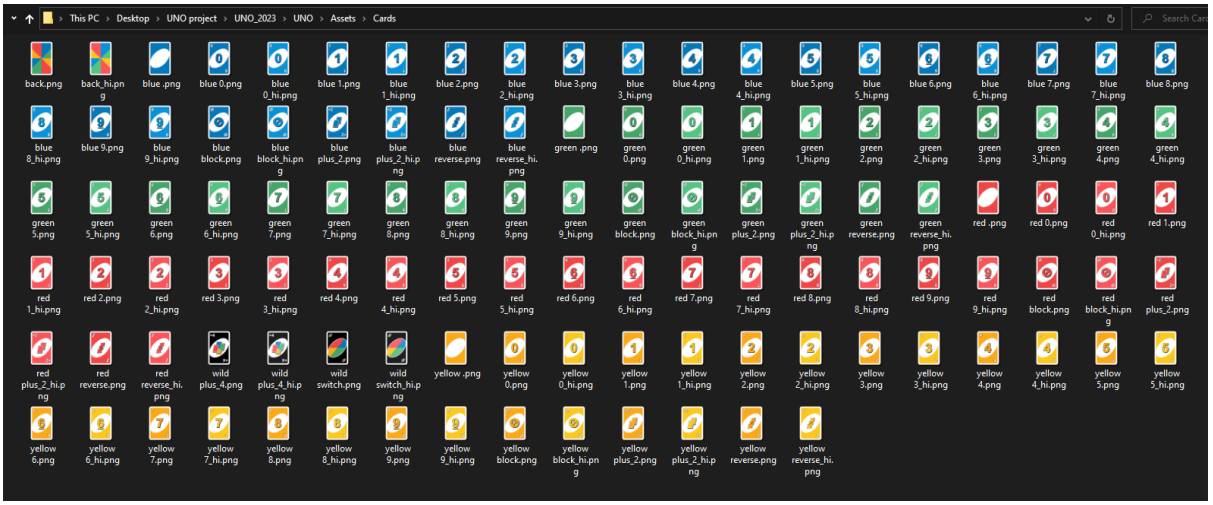
UNO



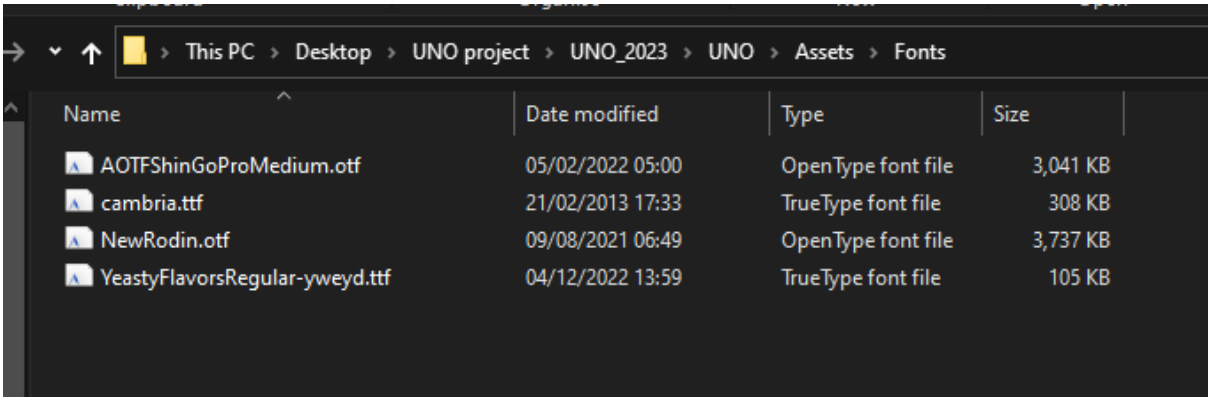
UNO/Assets



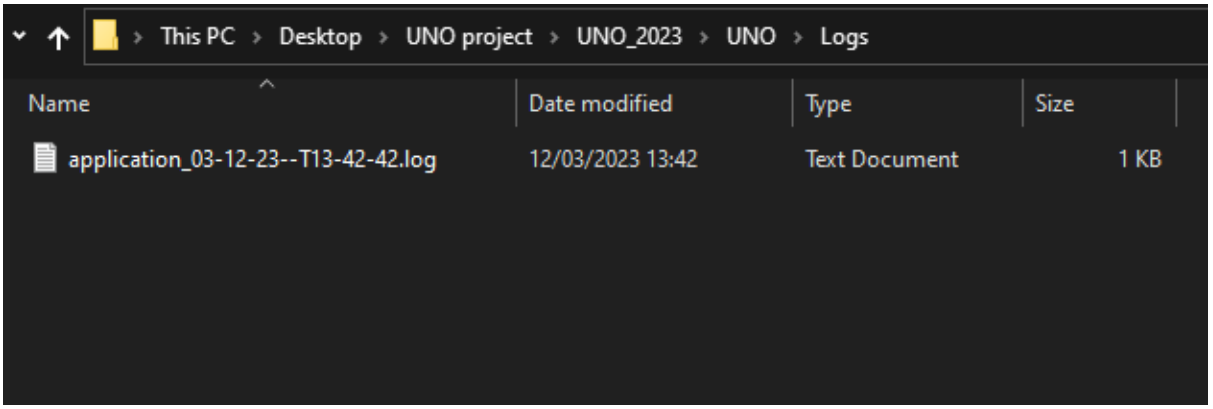
UNO/Assets/Cards



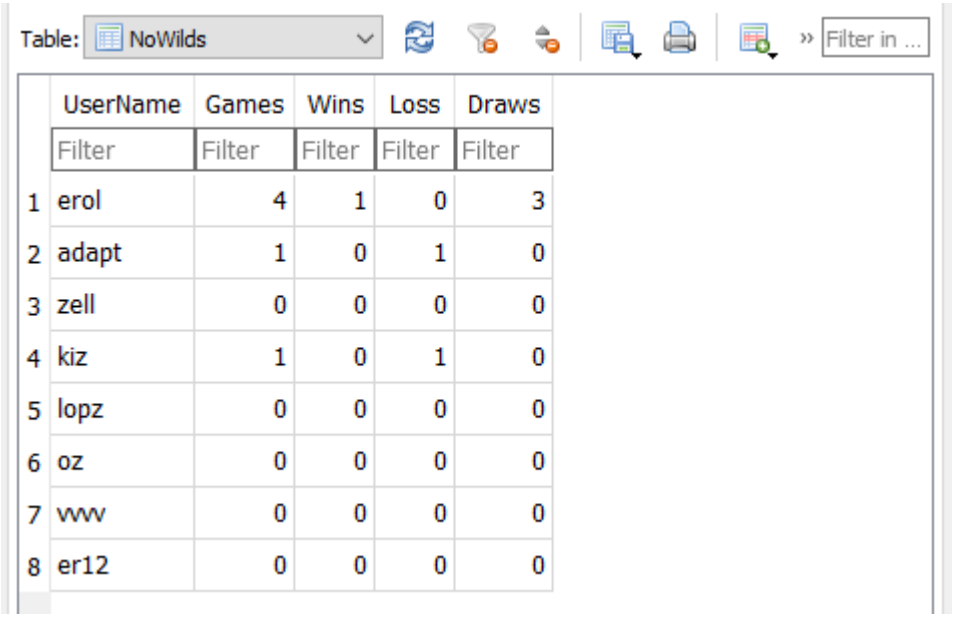
UNO/Assets/Fonts



UNO/Logs

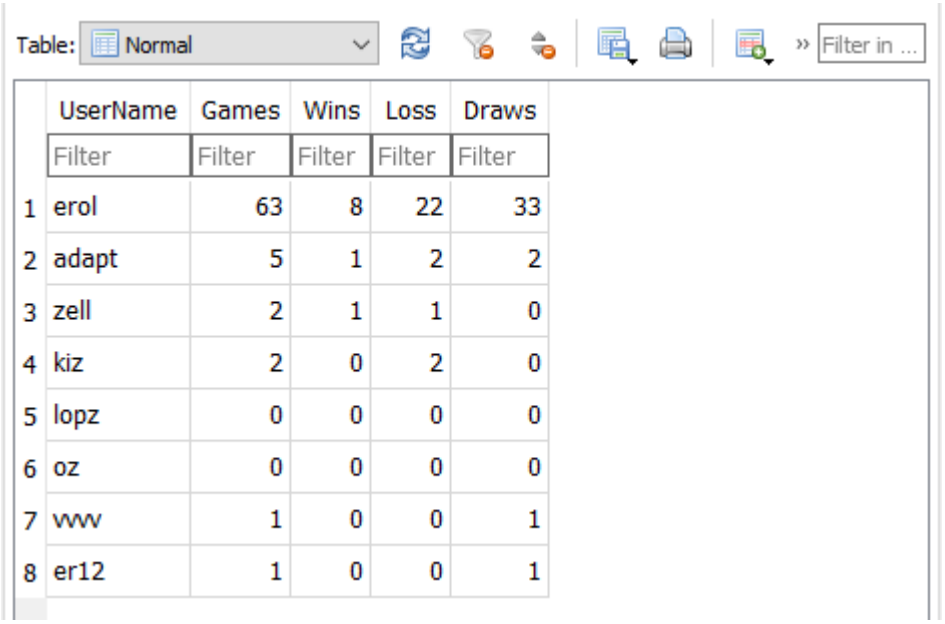


NoWilds table



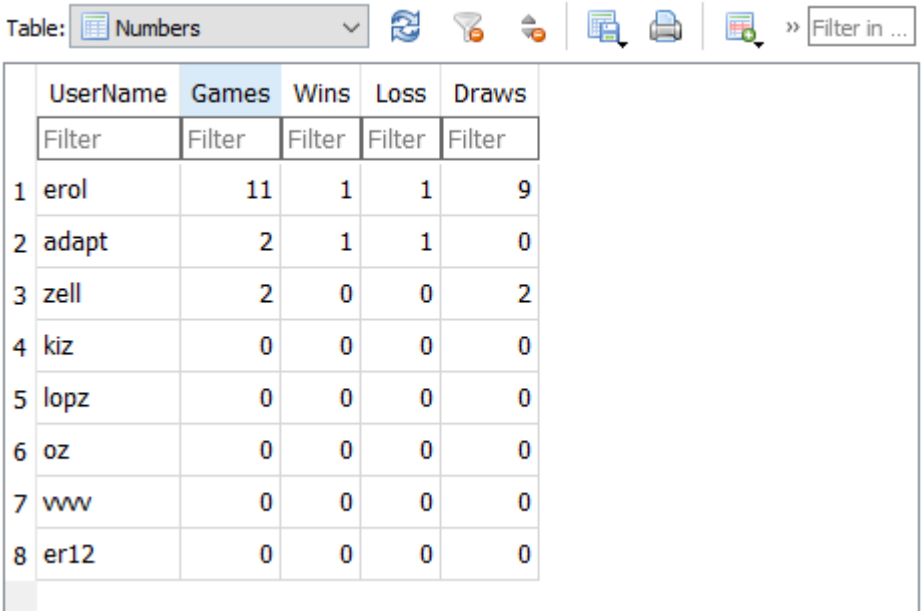
	UserName	Games	Wins	Loss	Draws
	Filter	Filter	Filter	Filter	Filter
1	erol	4	1	0	3
2	adapt	1	0	1	0
3	zell	0	0	0	0
4	kiz	1	0	1	0
5	lopz	0	0	0	0
6	oz	0	0	0	0
7	vvv	0	0	0	0
8	er12	0	0	0	0

Normal table



	UserName	Games	Wins	Loss	Draws
	Filter	Filter	Filter	Filter	Filter
1	erol	63	8	22	33
2	adapt	5	1	2	2
3	zell	2	1	1	0
4	kiz	2	0	2	0
5	lopz	0	0	0	0
6	oz	0	0	0	0
7	vvv	1	0	0	1
8	er12	1	0	0	1

Numbers table



	UserName	Games	Wins	Loss	Draws
	Filter	Filter	Filter	Filter	Filter
1	erol	11	1	1	9
2	adapt	2	1	1	0
3	zell	2	0	0	2
4	kiz	0	0	0	0
5	lopz	0	0	0	0
6	oz	0	0	0	0
7	vvv	0	0	0	0
8	er12	0	0	0	0

Final video of the application: To view use link:

https://youtu.be/sZ9qxBPcGJw

Testing

The goal of testing code is to ensure that code works properly and meets the requirements for which it was designed. Testing code is running a set of tests to ensure that it operates as intended and is free of faults or bugs that could cause it to fail or give incorrect results. Testing can also be used to condense and remove wasteful code, which can potentially slow down code.

Drive file that holds logs and videos to link: https://drive.google.com/drive/folders/1bI3v0uGI6sDRqef1AwrApcBZNQrTMepn?usp=share_link (When reading logs it advised you use software that displays lines for each separate log for example notepad ++ or VScode)

The Game Loop

The program must be able to set up a game with a selected game mode, complete checks and continue until a condition is met. The majority of tests target the main_game_uno function.

1. Each selected game mode choice is possible to be selected and saved to a variable choice from a mouse click on a button.
2. If a game mode is selected it returns the correct deck.
3. If the back button is selected during the game mode screen, the user returns to menu

4. Each player is to be dealt 7 cards at the start of the game from the correct deck
5. The starting card/first card cannot be a wild, so the first_card_check should continuously draw a new starting card until it is not a wild
6. If the user or AI player picks up a card, the algorithm should detect it (pick_up_card_used) and that should be replaced with a new card
7. If there are no more cards to pull from the deck, an IndexError should be purposely raised and the game to end
8. If a user or AI player plays plus 4 for a card, the algorithm should detect it (plus_4_check), then draw four new cards and call the append_cards function with all the new cards
9. If a user or AI player plays plus 2 for a card, the algorithm should detect it (plus_2_check), then draw two new cards and call the append_cards function with all the new cards
10. If an IndexError is called on purpose it should start the draw algorithm.
11. The draw algorithm should start a select SQL query where it should retrieve data from the selected game mode with the attributes games, draw data for the selected user and increment the data by one.
12. The draw algorithm after the selected data has been incremented, should run an sql update query to update the selected table with the incremented data in the correct column for the selected user
13. The iterative loop should stop if a win or loss has occurred
14. If the draw algorithm is finished, the algorithm should break out of the iterative loop of end_check.

Objective	Test	Result of Test
1) Each selected game mode choice is possible to be selected and saved to variable choice from a mouse click on a button.	This is can be seen on line 16 for Numbers, line 48 for NoWilds and line 81 for Normal application_03-09-23--T15-26-16.log	Successful
2) If a game mode is selected it returns the correct deck.	After the Number chosen on line 16, the deck on line 17 only has numbers. After Nowild is chosen on line 49, the deck on line 50 does not contain wilds. After the normal chosen on line 82, the deck on line 83 has all cards. application_03-09-23--T15-26-16.log	Successful

3) If the back button is selected during the game mode screen, the user returns to the menu	At lines 39 to 41. The user has pressed the back button and the application has purposely raised an index error application_03-09-23--T15-26-16.log	Successful
4) Each player is to be dealt 7 cards at the start of the game	From lines 19 to 27 the app draws cards for each player in the list. At line 36 we can see before anyone played yet, they all have 7 cards. At line 37 all players have the cards drawn from 19 to 27. application_03-09-23--T15-26-16.log	Successful
5) The starting card/first card cannot be a wild, so the first_card_check should continuously draw new starting cards until it is not a wild	In this test we altered the code to increase the number of wild cards in the normal deck to speed up the test. As seen on line 29 the first card is a wild switch. The algorithm keeps looping between line 30 to line 39 until a non-wildcard is chosen, as seen on line 39. application_03-09-23--T16-46-07.log	Successful
6) If the user or AI player picks up a card, the algorithm should detect it (pick_up_card_used) and that should be replaced with a new card	As seen at line 40 in the application log the player picks up the card yellow 4. At line 49 it detects a card has been picked up. At line 52 the card changes to blue 6 application log	Successfuthe l
7) If there are no more cards to pull from the deck, an IndexError should be purposely	In this text alter code was used to speed up the test. The user starts with 75 cards instead of 7. As seen on line 1001 where a plus 2 check is True, on line 1004 there are no	Successful

raised and the game to end	more cards to pull from the deck. The string “end” is used to end the game by raising an Index Error as seen on line 1016. application_03-09-23--T16-27-34.log	
8) If the user or AI player plays plus 4 for a card, the algorithm should detect it (plus_4_check), then draw four new cards and call the append_cards function with all the new cards	At line 361 player 4 plays plus 4. So for player 1 at line 364 plus 4 check is True. To 364 to 373 four cards are drawn. These cards at line 373 are added to a list. Then append_cards function called application log	Successful
9) If the user or AI player plays plus 2 for a card, the algorithm should detect it (plus_2_check), then draw two new cards and call the append_cards function with all the new cards	At line 472 player 2 plays yellow plus 2. So for player 3 at line 475 plus 4 checks is True. To 472 to 475 two cards are drawn. These cards are added to a list. Then append_cards function is called. Can be seen in the deck at 482 in player 3 deck application log	Successful
10) If an IndexError is called on purpose it should start the draw algorithm.	At line 2415 the user raises an index error that triggers the expectation. application log	Successful
11) The draw algorithm should start a select sql query where it should retrieve data from the selected game mode with the attributes games,	At line 42 the game ended with a draw. On line 43 we see that the select query has accessed in this example Numbers table data (games, draws) from where User = Erol.	Successful

draw data for the selected user and increment the data by one.	application_03-09-23--T15-26-16.log	
12) The draw algorithm after selected data has been incremented, should run an sql update query to update the selected table with the incremented data in the correct column for the selected user	At line 42 the game ended with a draw. On line 43 we see that the select query has accessed in this example Numbers table data (games, draws) from where User = Erol. The data is incremented by one as seen on lines 42 and 43. This data is then updated on the selected table. application_03-09-23--T15-26-16.log	Successful
13) The iterative loop should stop if a win or loss has occurred	At line 2384, the user has won a game. The Sql queries are finished and the application returns true to exit out of the end_check loop. This is seen on line 2387 as we are back on the main menu. application log	Successful
14) If the drawing algorithm is finished, the algorithm should break out of the iterative loop of end_check.	Line 41 a draw has occurred, it does the sql queries. The user returned back to the menu as seen on line 45. application_03-09-23--T15-26-16.log	Successful

Setting up app/Sign in and Sign up (start loop)

The application is to create a database that can store tables with multiple attributes to track user stats.

1. To create a database called “uno_database.db” if it doesn't exist already with all tables with their attributes.
2. User to be able to enter text through the GUI
3. Error codes are to be displayed on the GUI
4. The user cannot enter a username another person has used
5. Users can log in to the application by checking if the username is in the database
6. Cannot enter a blank username when signing up
7. When a Username is created and valid, all attributes for that user are set to 0 for all tables
8. The user username is in all tables
9. Appropriate error codes for invalid inputs

Objective	Test	Result of Test
1) To create a database called “uno_database.db” if it doesn't exist already with all tables with their attributes.	Can be seen on 2 to 4 lines of the log application_03-09-23--T18-15-05.log	Successful
2) User to be able to enter text through the GUI	This can be seen in the video at 00:00:31 2023-03-09 18-04-06.mp4	Successful
3) Error codes to be displayed on the GUI	This can be seen in the video at 00:00:48 2023-03-09 18-04-06.mp4	Successful
4) The user cannot enter a username another person has used for signing up	This can be seen in logs at line 37 to 39 and at 00:00:45 in the video. As we try to sign up with an already valid login. 2023-03-09 18-04-06.mp4 application_03-09-23--T18-04-09.log	Successful

5) User can log in to the application by checking if the username is in the database	This was tested on line 28 to line 31. As the user text for the login is found in the database 2023-03-09 18-04-06.mp4 application_03-09-23--T18-04-09.log	Successful
6) Cannot enter a blank username when signing up	Application does not allow the user to enter a blank text.	Successful
7) When a Username is created and valid, all attributes for that user are set to 0 for all tables	This was tested at line 7 from 00:00:32 to 00:00:38. When a user creates a new username it is inserted into all tables. 2023-03-09 18-04-06.mp4 application_03-09-23--T18-04-09.log	Successful
8) User username is in all tables	This was tested at 00:00:21 to 00:00:41 in the video. On the right hand of the video, we can see the user's username was added to all tables with the same stats. Also seen on line 7 2023-03-09 18-04-06.mp4 application_03-09-23--T18-04-09.log	Successful
9) Appropriate error codes for invalid inputs	This can be seen tested throughout the video and the logs at lines: 12,18,37 2023-03-09 18-04-06.mp4 application_03-09-23--T18-04-09.log	Successful

UNO Cards and Extra

The module is used to enforce rules against user choice, to create a digital recreation of Uno. The module is home to AI behaviour. The extra module is home to the sorting algorithm used for the program.

1. Players can play a card that shares the same colour or number/symbol as the top card
2. Players can play wild cards onto any top card.
3. User can pick up a card and not play it
4. The player can pick up and play it
5. The player that picks up a card and it, not either a wild or shares the same properties as the top card, is automatically added to the Player deck and nothing is played
6. A player plays a card that is not wild or shares the same properties as the top card, it is automatically added to the player's deck and nothing is played
7. The player can only pick up one card
8. The player can only play one card
9. The player can change from many different pages of cards during their go and not on their go
10. Each page should only show 7 cards at max
11. All cards need to be highlighted when hovered over
12. Each card needs to be clickable and playable
13. If an order is switched e.g clockwise to anticlockwise, it should continue until another player switches it and vice versa
14. Reverse cards when the player when clockwise is True, need to set it False and vice versa
15. If a block card is played it should skip the next player in order direction
16. All wild cards allow the user to change the colour to either (red. green, yellow, or blue)
17. Plus four cards need to add four additional cards to the next player deck (append cards)
18. Plus two cards need to add four additional cards to the next player deck (append cards)
19. All cards played are in the players' deck and are removed after they have played it
20. If a player runs out of cards in their deck, the end checks to be called
21. If the User runs out of cards in their deck calling the end check, it should use a sql select statement to increment the user wins and games and update them in table
22. If an AI player runs out of cards in their deck calling the end check, it should use a sql select statement to increment the user losses and games and update them in table
23. After each goes the next_turn function should be called to go to the next player depending on the order direction.
24. Each game should have random order of cards in each play hand and in deck
25. The user should not be able to play a card while it not their turn
26. The AI cards should be iterated through with weights being added to it from the dictionary _cards_weight and added to list_for_cards
27. A lambda function is called to remove all non-playable cards from the AI player deck compared to the top card
28. All colours from the AI player deck after the non-playable cards are removed should be counted, so they can be added on top of the card's current weight
29. All cards are to be recursively sorted in order of weight using quick sort, after the new addition of more weights for the AI player
30. During AI player go position state should be called when deciding how aggressively the Ai player is going to play
31. AI aggression should be 2 if it loses compared to the next player
32. AI aggression should be 0 if it drawing compared to the next player

33. AI aggression should be -2 if it wins compared to the next player
34. AI player can play it selected card
35. If AI has no playable cards it should pick up and attempt to play if the picked card is not compatible it would be added to their deck
36. If an AI player decides to play a wild, it should get all the AI player's card weights and count how many colours the player has in hand, add them together to get the final weight and sort them using merge sort.
37. The player after the merge sort has a 13/20 chance to play the best colour or if he has another colour deck and 7/20 to play their second best colour.

Objective	Test	Result of Test
1) Players can play a card that shares the same colour or number/symbol as the top card	This test can be seen on line 43 of the log and in the video at 00:00:29. The card played shares the same colour. This can also be seen by player 2 at line 441 in the log and in the video at 00:00:54, as the player plays a plus 2 card that shares the same symbol. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
2) Players can play wild cards onto any top card.	This test can be seen at line 997,999,1000,1023 and 1042 in the logs as the player plays the card on top of a yellow 5, changes the colour to red and plus four are removed from the deck. Can be seen in the video at 00:01:50 to 00:01:54 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
3) User can pick up a card and not play it	This test can be seen in logs at lines 383 to 392 as the user picks up a card and does not play it. This can be seen in the video from 00:00:50 to	Successful

	00:00:53. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	
4) Player can pick up and play the card	This can be seen in the logs at (line: 1847, 1848-1850,1859) as the AI player 3 picks up the card blue 2 and plays it with the same properties as yellow 2. This can be seen in the video at 00:02:39 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
5) Player that picks up a card and it is not either a wild or share the same properties as the top card, it is automatically added to the Player deck and nothing played	This test can be seen in the logs at line 215 to 220 as the user picks up the card and plays it. This can further be seen in the video at 00:00:41 to 00:00:44 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
6) Player can only pick up one card	This can be seen in the video at 00:00:42 as the player picks up a yellow 8. This can be seen in the logs from lines 215 to 224 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
7) Player can only play one card	This can be seen at 00:00:30 in the video as the player plays one card (red 3). This can be seen in the logs at line 42 application_03-10-23--T13-4	Successful

	7-48 2023-03-10 13-47-44.mp4	
8) Player can change from many different pages of cards during their go and not on their go	This can be seen at 00:01:23 in the video as the user flips through pages 1 and 2, to check his cards. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
9) Each page should only show 7 cards at max	This can be seen at 00:01:23 in the video as the user flips through pages 1 and 2, we can see that on page 1 there are only 7 cards on the screen before going onto the next page. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
10) All cards need to be highlighted when hovered over (non-including display loop)	This test can be seen throughout the video anytime the user hovers over a clickable object. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
11) Each card needs to be clickable and playable (non-including display loop)	This test can be seen throughout the video anytime the user wants to play a card in the play loop. The user clicks the card and it played application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful

12) If an order is switched e.g clockwise to anticlockwise, it should continue until another player switches it and vice versa	This can be seen in the video at 00:03:02 to 00:03:20 as player four plays a reverse that changes the order to be clockwise and then the user plays another reverse card that switches the order back. This can also be seen in the log at lines 2209,2210,2218,2228,2229 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
13) Reverse cards when the player when clockwise is True, need to set it False and vice versa	This can be seen in the video at 00:03:02 to 00:03:08 as player four plays a reverse that changes the order to be clockwise and then the user plays another reverse card that switches the order back. This can also be seen in the log at lines 2209,2210,2218,2228,2229 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
14) If the block card is played it should skip the next player in order direction	This can be seen in the video from 00:01:29 to 00:01:32. This as the user plays a blue block that skips player 2 turns, (as the direction is clockwise). This can be seen in the logs at line 734,752 (player 2 turns skipped at 753 to 755) and line 785. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful

15) All wild cards allow the user to change the colour to either (red. green, yellow, or blue)	This test can be seen at lines 997,999,1000,1023,1042 and in the logs as the player plays the card on top of a yellow 5, changes the colour to red and plus four are removed from the deck. Can be seen in the video at 00:01:50 to 00:01:54 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
16) Plus four cards need to add four additional cards to the next player deck (append cards)	This test can be seen at lines 997,999,1000,1023,1026,1035-1037 and 1042 this in the logs as the player plays the card on top of a yellow 5, changing the colour to red from the selection of colours. Can be seen in the video at 00:01:50 to 00:01:54 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
17) Plus two cards need to add two additional cards to the next player's deck (append cards)	This can be seen in the video at 00:00:48 as player 4 plays plus 2. This can be seen in more detail in the logs as at line 366 the card played, throughout lines 370 to 376, two cards are drawn and added to player 1 deck. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
18) All cards played are in the player's deck and are removed after they have played it	This test can be seen throughout the video and logs. An example is at 00:00:30 in the video as the player plays one card (red 3) and the card is seen to be removed from	Successful

	the deck. This can be seen in the logs at lines 42 to 55 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	
19) If a player runs out of cards in their deck, the end check is to be called	This was tested at 00:03;19 when the user played his last card in the game. This is further seen in the logs at 2516, 2527-2530 application_03-10-23--T13-47-48 2023-03-10 19-00-11.mp4	Successful
21)If the User runs out of cards in their deck calling the end check, it should use a sql select statement to increment the user wins and games and update them in the table	This can be seen after the win at 00:03;19, I then refresh the table to see updated stats, and both wins and games are incremented only in that table. This is further seen in the logs from 2528 to 2530. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
22) If an AI player runs out of cards in their deck calling the end check, it should use a sql select statement to increment the user losses and games and update them in the table	This test can be seen by the logs, as when player 2 plays red 0, his last card (line 2559) he triggers the game to end (line 2563-2566). Zell/User losses and games play stats are incremented by one as seen on the logs. This can further be seen on the video from 00:02:10 to the end, where player 2 plays his final card and the game ends, and the User stats are updated in the table	Successful

	application_03-10-23--T18-59-55 2023-03-10 19-00-11.mp4	
23) After each turn the next_turn function should be called to go to the next player depending on the order direction.	This can be seen throughout the video after each player's turn. It can be seen throughout logs too e.g. line 56 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
24) Each game should have random order of cards in each play hand and in the deck	This can be seen throughout the video for the user's hand, but this can be seen in more detail through logs at line 37,20 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
25) User should not be able to play a card while it is not their turn	This can be seen throughout the video, but an example is at 00:00:28 to 00:00:33 where the User tries to play his card, but nothing is played 2023-03-10 19-00-11.mp4	Successful
26) The AI cards should be iterated through with weights being added to it from the dictionary _cards_weight and added to list_for_cards	This can be seen throughout the logs. An example of it is lines 64 to 76. application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
27) A lambda function is called to remove all non-playable cards from the AI player deck compared to the top card	This can be seen through the logs. An example of this is seen in the logs at lines 78 to 79 application_03-10-23--T13-4	Successful

	7-48 2023-03-10 13-47-44.mp4	
28) All colours from the AI player deck after the non-playable cards are removed should be counted, so they can be added on top of the card's current weights	This can be seen through the logs. An example of this is seen in lines 81 to 82 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
29) All cards are to be recursively sorted in order of weight using quick sort, after the new addition of more weights for the AI player	This can be seen through the logs. An example of this is seen on lines 82 to 83 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
30) During the AI player's turn, the position state should be called when deciding how aggressively the Ai player is going to play	This can be seen through the logs. An example of this is seen in line 84 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
31) AI aggression should be 2 if it losing compared to the next player	This can be seen through the logs. An example of this is seen in lines 345 to 348 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
32) AI aggression should be 0 if it drawing compared to the next player	This can be seen through the logs. An example of this is seen in lines 84 to 87 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful

33) AI aggression should be -2 if it winning compared to the next player	This can be seen through the logs. An example of this is seen in lines 420 to423 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
34) AI player can play a selected card	This can be seen throughout our video but further can be seen in the logs at 93 where player 2 wants to play a red 9 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
35) If AI has no playable cards it should pick up and attempt to play as if the picked card is not compatible it would be added to their deck	This can be seen in the logs at (line: 2025 - 2035, 2042) as the AI player picks up green plus_2, but because it didn't share any properties with the top card (blue 0), it added to player deck further seen at line 2041 application_03-10-23--T13-47-48 2023-03-10 13-47-44.mp4	Successful
36) If an AI player decides to play a wild, it should get all AI players card weights and count how many colours the player has in hand, add them together to get the final weight and sort them using merge sort.	As seen on the log at line 147 where player 3 plays a wild switch. During lines 147 to 150. The weights are added and sorted. application_03-10-23--T18-59-55 2023-03-10 19-00-11.mp4	Successful
37) The player after the merge sort has a 13/20 chance to play the best colour or if he	As seen on the log at line 147 where player 3 plays a wild switch. During lines 147 to 151. The weights are added	Successful

has another colour deck and 7/20 to play their second best colour.	and sorted. The player chooses the best colour and the card is turned green as seen on 169 application_03-10-23--T18-59-55 2023-03-10 19-00-11.mp4	

Evaluation

Objectives Completed

Objectives	Completed
• The application needs a login and sign-up system that is attached to a table that uses wins/losses/draws/games as its attributes.	Implemented

• A database that is created if it doesn't exist	Implemented
• All menu and options/gameplay must be displayed on a GUI to restrict user inputs and to create a more user-oriented experience	Implemented
• The program needs a menu to allow users to play the game, view their stats and quit after they're done. This should be done with the use of buttons that are intractable with a mouse.	Implemented
• The game needs three AI players that are able to play rationally and are able to determine their options to be able to play the best card possible, but with a handicap of a bit chance to make AI players less predictable.	Implemented
• The AI players need to be able to draw cards, and play cards	Implemented
• Have dynamic aggression depending on if there winning or losing compared to the player next to them	Implemented
• During the game there would need to be a clockwise and anticlockwise order, that can be switch if needed	Implemented

• All normal playing cards and symbol/special cards need to be fully working and integrated into the game	Implemented
• The game need a hub menu where it clicks way to start a game, view yours and other stats and quit the game	Implemented
• The game needs win/draw/loss conditions that when triggered tell the user if they won/loss/drawn and update their stats	Implemented
• The player needs to see all his cards in easy and fast way to be able to decide which card he wants to play with the uses of multiple pages to keep the screen less cluttered	Implemented
• The player need to be able to play, pick up all types of cards if they have a move available	Implemented
• The user needs to be punished for playing cards that are not playable compared with the top card. This would be done by the user being forced to pick up and not able to play the card	Implemented
• The player needs to have the option when picking up to either play the card he picked up or play the card. However, a special	Implemented

condition needs to be made to void the player move/the card played if it can't be played compared to the top card, however, without punishment	
• The game needs a huge amount of assets either made by me with the use of adobe photoshop or downloaded with a licence. The assets that are needed for the game are buttons, cards, fonts, backgrounds and ect.	Implemented

Feedback

Ismail interview

Me: " Ive implemented both the login and sign-up system, as well as a GUI that displays all the game's options and rules. How did you find playing the game in general though?"

Ismail: "I really enjoyed playing the game! The AI players were challenging, but not impossible to beat. I liked that the game had a variety of options, such as viewing my stats and starting a new game. Overall, it was a great user experience."

Me: "What do you think can be improved in the game?"

Ismail: "I think the game could benefit from having sound effects and more animations to make it more engaging. Also, having more customization options for the players would make the game more engaging. It would be great if the game could have a multiplayer mode so that I can play with my friends."

Me: "That's good to hear! What did you think of the gameplay and rules?"

Ismail: "The gameplay was smooth and the rules were easy to follow. I liked that you mixed official rules with house rules, it made the game more interesting."

James Interview

Me: "Did you notice anything that could be improved,James?"

James: "The only thing I can think of is that the game could use some additional sound effects. Maybe some music or ambient noise in the background would add to the overall experience.":

Me: "Thanks for your feedback, James. Would you recommend the game to others?"

James: "Yeah absolutely! It's a fun and engaging card game that I think anyone would enjoy."

Me: "What did you think about the difficulty level of the AI players?"

James: "I thought the difficulty level was just right. The AI players were not too predictable, which made the game more exciting. However, they were not too difficult to the point where the game was frustrating. Overall, the difficulty level was balanced."

Me: "What would you suggest for future updates to the game?"

James: "I would suggest adding a multiplayer mode so that players can play with their friends. . It would be cool to see more types of special cards added to the game, for example the switch hands card. Lastly, adding achievements and rewards for completing certain objectives could make the game more engaging."

A interview with both james and ismail

Me: "Thanks for the feedback! Did you encounter any issues while playing?"

Ismail: "No, not really. Everything worked smoothly and there were no bugs or glitches."

James: "Same here, I didn't experience any issues. It was a really fun game to play!"

Me: "That's good to hear! Do you have any suggestions for improvements or new features?"

Ismail: "I think it would be cool to have an online multiplayer feature, so that we can play with friends who are far away."

James: "Yeah, that would be awesome! I also think it would be cool to have different themes for the cards and background."

Me: "Anything else you'd like to add?"

Ismail: "Overall, I really enjoyed playing this game. It was easy to use and had a nice design. I would definitely play it again!"

James: "Same here! I had a lot of fun playing and I think you did a great job with the game."

Me: "Thank you both for the feedback! It was really helpful in knowing what I should do next time and I'm happy that you enjoyed the game. Are you sure there aren't any other suggestions you guys have in terms of the user interface or gameplay?"

Ismail: "One thing I noticed during gameplay is that it can be a bit confusing when there are multiple pages of cards. Maybe there could be some sort of indication to show which page the player is on."

Improvements for the future

As Ismail stated I think the game could "benefit from having sound effects and more animations" or as James said some "ambient background music". It would make the game more "engaging" by just filling the void of emptiness at the moment. This could be done with the playsound module for python. Another thing could be a scalable window if I wrote an algorithm that gets the width and height of a window and uses it to rescales coords, images and text. I think a good idea for the future would be to add local and "online multiplayer" with the ability to choose how many players you want to play and if not all slots are filled the AI players will be added instead. Maybe another thing could be even more game modes e.g "timed mode" as James said before or a way to let users change the backgrounds with custom images. A minor tweak as ismail said is to add an indication to tell what page you are on.